

RTL Design Sherpa

RTL AMBA AXI5-Lite Module Reference — Rev 0.92

October 2026

Table of Contents

1 AXI5-Lite (AXIL5) Modules....15
1.1 Overview....16
1.2 Functional Description....17
1.3 Usage Example....19
1.4 Design Notes....20
1.5 Related Modules....21
1.5.1 Transport (4)....21
1.5.2 Clock-gated (4)....21
1.5.3 Monitored (4)....21
1.5.4 Monitored and clock-gated (4)....22
1.6 Testing....23
1.7 References....24
1.8 Navigation....26
2 AXIL5 Master Read....27
2.1 Overview....28
2.2 Parameters....29
2.3 Ports....31
2.4 Functional Description....33
2.4.1 Block diagram....33
2.4.2 Optional signal groups....33
2.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....34
2.4.4 It transports; it does not interpret....34

2.5 Timing Characteristics....36
2.5.1 Buffer Depths and Latency....36
2.5.2 Optional-group effect....36
2.6 Usage Examples....37
2.7 Design Notes....38
2.8 Related Modules....39
2.9 Testing....40
2.10 References....41
2.11 Navigation....42
3 AXIL5 Master Read, Clock-Gated....43
3.1 Overview....44
3.2 Parameters....45
3.3 Ports....47
3.4 Functional Description....49
3.4.1 Block diagram....49
3.4.2 Optional signal groups....49
3.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....50
3.4.4 It transports; it does not interpret....50
3.5 Timing Characteristics....52
3.5.1 Buffer Depths and Latency....52
3.5.2 Optional-group effect....52
3.6 Usage Examples....53
3.7 Design Notes....54
3.8 Related Modules....55

3.9 Testing....56
3.10 References....57
3.11 Navigation....58
4 AXIL5 Master Read Monitor....59
4.1 Overview....60
4.2 Parameters....61
4.3 Ports....65
4.4 Functional Description....70
4.4.1 Block diagram....70
4.4.2 Optional signal groups....70
4.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....71
4.4.4 It transports; it does not interpret....71
4.5 Design Notes....73
4.6 Timing Characteristics....74
4.6.1 Buffer Depths and Latency....74
4.6.2 Optional-group effect....74
4.7 Usage Examples....75
4.8 Related Modules....77
4.9 Testing....78
4.10 References....79
4.11 Navigation....80
5 AXIL5 Master Read Monitor, Clock-Gated....81
5.1 Overview....82
5.2 Parameters....83

5.3 Ports....86
5.4 Functional Description....92
5.4.1 Block diagram....92
5.4.2 Optional signal groups....92
5.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....93
5.4.4 It transports; it does not interpret....93
5.5 Design Notes....95
5.6 Timing Characteristics....96
5.6.1 Buffer Depths and Latency....96
5.6.2 Optional-group effect....96
5.7 Usage Examples....97
5.8 Related Modules....99
5.9 Testing....100
5.10 References....101
5.11 Navigation....103
6 AXIL5 Master Write....104
6.1 Overview....105
6.2 Parameters....106
6.3 Ports....108
6.4 Functional Description....110
6.4.1 Block diagram....110
6.4.2 Optional signal groups....110
6.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....111
6.4.4 It transports; it does not interpret....111

6.5 Timing Characteristics....113
6.5.1 Buffer Depths and Latency....113
6.5.2 Optional-group effect....113
6.6 Usage Examples....114
6.7 Design Notes....115
6.8 Related Modules....116
6.9 Testing....117
6.10 References....118
6.11 Navigation....119
7 AXIL5 Master Write, Clock-Gated....120
7.1 Overview....121
7.2 Parameters....122
7.3 Ports....124
7.4 Functional Description....126
7.4.1 Block diagram....126
7.4.2 Optional signal groups....126
7.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....127
7.4.4 It transports; it does not interpret....127
7.5 Timing Characteristics....129
7.5.1 Buffer Depths and Latency....129
7.5.2 Optional-group effect....129
7.6 Usage Examples....130
7.7 Design Notes....132
7.8 Related Modules....133

7.9 Testing....134
7.10 References....135
7.11 Navigation....136
8 AXIL5 Master Write Monitor....137
8.1 Overview....138
8.2 Parameters....139
8.3 Ports....143
8.4 Functional Description....149
8.4.1 Block diagram....149
8.4.2 Optional signal groups....149
8.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....150
8.4.4 It transports; it does not interpret....150
8.5 Design Notes....152
8.6 Timing Characteristics....153
8.6.1 Buffer Depths and Latency....153
8.6.2 Optional-group effect....153
8.7 Usage Examples....154
8.8 Related Modules....156
8.9 Testing....157
8.10 References....158
8.11 Navigation....159
9 AXIL5 Master Write Monitor, Clock-Gated....160
9.1 Overview....161
9.2 Parameters....162

9.3 Ports....165
9.4 Functional Description....171
9.4.1 Block diagram....171
9.4.2 Optional signal groups....171
9.4.3 With no ID and every group disabled, this IS the AXI4-Lite module....172
9.4.4 It transports; it does not interpret....172
9.5 Design Notes....174
9.6 Timing Characteristics....175
9.6.1 Buffer Depths and Latency....175
9.6.2 Optional-group effect....175
9.7 Usage Examples....176
9.8 Related Modules....178
9.9 Testing....179
9.10 References....180
9.11 Navigation....182
10 AXIL5 Slave Read....183
10.1 Overview....184
10.2 Parameters....185
10.3 Ports....187
10.4 Functional Description....189
10.4.1 Block diagram....189
10.4.2 Optional signal groups....189
10.4.3 With no ID and every group disabled, this IS the AXI4-Lite module..190
10.4.4 It transports; it does not interpret....190

10.5 Timing Characteristics....192
10.5.1 Buffer Depths and Latency....192
10.5.2 Optional-group effect....192
10.6 Usage Examples....193
10.7 Design Notes....194
10.8 Related Modules....195
10.9 Testing....196
10.10 References....197
10.11 Navigation....198
11 AXIL5 Slave Read, Clock-Gated....199
11.1 Overview....200
11.2 Parameters....201
11.3 Ports....203
11.4 Functional Description....205
11.4.1 Block diagram....205
11.4.2 Optional signal groups....205
11.4.3 With no ID and every group disabled, this IS the AXI4-Lite module..206
11.4.4 It transports; it does not interpret....206
11.5 Timing Characteristics....208
11.5.1 Buffer Depths and Latency....208
11.5.2 Optional-group effect....208
11.6 Usage Examples....209
11.7 Design Notes....210
11.8 Related Modules....211

11.9 Testing....212
11.10 References....213
11.11 Navigation....214
12 AXIL5 Slave Read Monitor....215
12.1 Overview....216
12.2 Parameters....217
12.3 Ports....221
12.4 Functional Description....226
12.4.1 Block diagram....226
12.4.2 Optional signal groups....226
12.4.3 With no ID and every group disabled, this IS the AXI4-Lite module..227
12.4.4 It transports; it does not interpret....227
12.5 Design Notes....229
12.6 Timing Characteristics....230
12.6.1 Buffer Depths and Latency....230
12.6.2 Optional-group effect....230
12.7 Usage Examples....231
12.8 Related Modules....233
12.9 Testing....234
12.10 References....235
12.11 Navigation....236
13 AXIL5 Slave Read Monitor, Clock-Gated....237
13.1 Overview....238
13.2 Parameters....239

13.3 Ports....242
13.4 Functional Description....248
13.4.1 Block diagram....248
13.4.2 Optional signal groups....248
13.4.3 With no ID and every group disabled, this IS the AXI4-Lite module..249
13.4.4 It transports; it does not interpret....249
13.5 Design Notes....251
13.6 Timing Characteristics....252
13.6.1 Buffer Depths and Latency....252
13.6.2 Optional-group effect....252
13.7 Usage Examples....253
13.8 Related Modules....255
13.9 Testing....256
13.10 References....257
13.11 Navigation....259
14 AXIL5 Slave Write....260
14.1 Overview....261
14.2 Parameters....262
14.3 Ports....264
14.4 Functional Description....266
14.4.1 Block diagram....266
14.4.2 Optional signal groups....266
14.4.3 With no ID and every group disabled, this IS the AXI4-Lite module..267
14.4.4 It transports; it does not interpret....267

14.5 Timing Characteristics....269
14.5.1 Buffer Depths and Latency....269
14.5.2 Optional-group effect....269
14.6 Usage Examples....270
14.7 Design Notes....271
14.8 Related Modules....272
14.9 Testing....273
14.10 References....274
14.11 Navigation....275
15 AXIL5 Slave Write, Clock-Gated....276
15.1 Overview....277
15.2 Parameters....278
15.3 Ports....280
15.4 Functional Description....282
15.4.1 Block diagram....282
15.4.2 Optional signal groups....282
15.4.3 With no ID and every group disabled, this IS the AXI4-Lite module..283
15.4.4 It transports; it does not interpret....283
15.5 Timing Characteristics....285
15.5.1 Buffer Depths and Latency....285
15.5.2 Optional-group effect....285
15.6 Usage Examples....286
15.7 Design Notes....288
15.8 Related Modules....289

15.9 Testing....290
15.10 References....291
15.11 Navigation....292
16 AXIL5 Slave Write Monitor....293
16.1 Overview....294
16.1.1 Key Features....294
16.2 Parameters....295
16.3 Ports....299
16.4 Functional Description....305
16.4.1 Block diagram....305
16.4.2 With no ID and every group disabled, this IS the AXI4-Lite module..306
16.4.3 It transports; it does not interpret....306
16.5 Design Notes....308
16.6 Timing Characteristics....309
16.6.1 Buffer Depths and Latency....309
16.6.2 Optional-group effect....309
16.7 Usage Examples....310
16.8 Related Modules....312
16.9 Testing....313
16.10 References....314
16.11 Navigation....315
17 AXIL5 Slave Write Monitor, Clock-Gated....316
17.1 Overview....317
17.1.1 Key Features....317

17.2 Parameters....318
17.3 Ports....321
17.4 Functional Description....327
17.4.1 Block diagram....327
17.4.2 With no ID and every group disabled, this IS the AXI4-Lite module..328
17.4.3 It transports; it does not interpret....328
17.5 Design Notes....330
17.6 Timing Characteristics....331
17.6.1 Buffer Depths and Latency....331
17.6.2 Optional-group effect....331
17.7 Usage Examples....332
17.8 Related Modules....334
17.9 Testing....335
17.10 References....336
17.11 Navigation....338

1 AXI5-Lite (AXIL5) Modules

Location: `rtl/amba/axil5/` **Sibling family:** AXI4-Lite

1.1 Overview

Sixteen modules mirroring the AXI4-Lite family one for one, with the AXI5-Lite transaction ID and optional signal groups threaded through. Same channel set, same handshakes, same single-beat semantics – what AXI5-Lite adds is the ID and sideband (IHI 0022 Issue L, B2.1.5 and Table B2.2).

If you came here looking for a full AXI5-Lite protocol stack, read the next section before you instantiate anything.

1.2 Functional Description

These modules **transport** AXI5-Lite signals. They do not execute AXI5-Lite semantics:

- The ID (AWID/BID, ARID/RID) is carried in both directions; reflecting AxID onto BID/RID is the Subordinate's job (A5.1.1), and responses to different IDs may be reordered by it (B2.1.5).
- USER and TRACE are carried end to end unmodified.
- POISON is carried. It is never generated and never checked.

Those behaviours belong to the endpoints on either side. Read this before treating the family as a full AXI5-Lite protocol stack – it is not one.

AXI4-Lite vs AXI5-Lite, as this library implements them:

Feature	AXI4-Lite	AXI5-Lite
Channels	AW, W, B, AR, R	same
Burst support	none	none
Data width	32 or 64 (IHI 0022H B1.1.2)	any AXI width, 8..1024 (L Table A3.19; Table B2.4 adds no Lite limit)
Transaction ID	none	`AWID`/`BID`, `ARID`/`RID`, `ID_WIDTH` bits (L Table A5.2: 0..32). Optional on a Manager, mandatory on a Subordinate (Table B2.2, YS), so `ID_WIDTH` may be 0 on the `axil5_master_*` modules and must be >= 1 on the `axil5_slave_*` ones
User sideband	not present	`AxUSER`/`WUSER`/`BUSER`/ `RUSER`, gated by `ENABLE_USER`
Trace	not present	`AxTRACE`/`BTRACE`/`RTRACE`, gated by `ENABLE_TRACE`
Poison	not present	`WPOISON`/`RPOISON`, one bit per 64 data bits, gated by `ENABLE_POISON`

Feature	AXI4-Lite	AXI5-Lite
Exclusive access	not present	not present either (B2.1.5); no `AxLOCK`
MPAM / MECID / NSAID / Loopback	not present	not present either (Table B2.2 “N”)

Both data width and `ID_WIDTH` are checked at elaboration by every module’s `param_check` block; an illegal value stops the build.

With no ID and every group disabled the two are the same interface. On the Manager-side modules (`ID_WIDTH = 0`, every `ENABLE_*` = 0) the packed payload widths match channel for channel, which is what lets one testbench bind to either family – `val/amba/test_axil5_axil4_equivalence.py` pins it.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP were carried behind `ENABLE_*` parameters until issue #102; Table B2.2 marks them not present on AXI5-Lite, so they were removed, and the ID the table requires was added.

1.3 Usage Example

Two configurations – a Manager-side module configured down to AXI4-Lite, and a Subordinate-side module with its ID and sideband:

```
// AXI4-Lite-equivalent Manager side: no ID, every group off --
// payload widths identical to axil4_master_rd
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .ID_WIDTH        (0),
    .ENABLE_USER (0), .ENABLE_TRACE (0), .ENABLE_POISON (0)
) u_plain ( /* ... */ );

// Subordinate side with IDs (mandatory here) and the trace/user
sideband
axil5_slave_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (64),
    .ID_WIDTH        (6),
    .USER_WIDTH      (8),
    .ENABLE_USER (1), .ENABLE_TRACE (1), .ENABLE_POISON (1)
) u_sub ( /* ... */ );
```

1.4 Design Notes

Known limitations:

- No protocol checking of the ID or the optional groups. The monitored variants observe handshakes, addresses, responses and timing; since amba #104 their monitor also sees AxID and BID/RID, so responses a Subordinate reorders across IDs are paired with their own transactions (W in AW order). `axi_monitor_filtered` has no ports for USER, TRACE or POISON.
- No poison generation or checking.
- Timing diagrams are not yet drawn for this family.

1.5 Related Modules

The family, grouped by what each variant adds:

1.5.1 Transport (4)

Module	Channels
axil5_master_rd	AR, R
axil5_master_wr	AW, W, B
axil5_slave_rd	AR, R
axil5_slave_wr	AW, W, B

1.5.2 Clock-gated (4)

Module	Adds
axil5_master_rd_cg	One `amba_clock_gate_ctrl` over the whole inner module
axil5_master_wr_cg	One `amba_clock_gate_ctrl` over the whole inner module
axil5_slave_rd_cg	One `amba_clock_gate_ctrl` over the whole inner module
axil5_slave_wr_cg	One `amba_clock_gate_ctrl` over the whole inner module

1.5.3 Monitored (4)

Module	Adds
axil5_master_rd_mon	`axi_monitor_filtered`, monbus packet output
axil5_master_wr_mon	`axi_monitor_filtered`, monbus packet output
axil5_slave_rd_mon	`axi_monitor_filtered`, monbus packet output
axil5_slave_wr_mon	`axi_monitor_filtered`, monbus packet output

1.5.4 Monitored and clock-gated (4)

Module	Adds
axil5_master_rd_mon_cg	Both, with ready held low while gated

Module	Adds
axil5_master_wr_mon_cg	Both, with ready held low while gated
axil5_slave_rd_mon_cg	Both, with ready held low while gated
axil5_slave_wr_mon_cg	Both, with ready held low while gated

1.6 Testing

`val/amba/test_axil5_*.py` – sixteen runners, 42 parameterised cases, all passing. They drive the modules with **every optional group enabled**, which makes them the only tests in the repo that exercise the AXI5-Lite sideband against real transport RTL rather than a behavioural model.

The testbench classes in `bin/TBClasses/axil5/` are the AXI4-Lite ones with the component factories swapped and the group widths set in `COMPONENT_KWARGS`. One definition of the traffic and its checks.

Also here: `test_axil5_opt_signals.py` drives `axil5_opt_slave`, a behavioural slave under `rtl/amba/axil5/test-modules/` built purely so the BFMs had a DUT carrying every optional port. It is test collateral – do not instantiate it in a design.

1.7 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR/R and AW/W/B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): the family carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19, ID width per Table A5.2.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: storage on a back-pressured channel adds latency without changing function.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake

Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).

- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI”, §8.3 “AXI4 interfaces (full and AXI-Lite)” – textbook treatment of the AXI-Lite channel set (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`); clock-gating papers in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

1.8 Navigation

AMBA overview | AXI4-Lite | AXI5

2 AXIL5 Master Read

Module: `axil5_master_rd.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd`

2.1 Overview

The AXIL5 Master Read provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled

2.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
ARSize	int	AW + 3 +	

Parameter	Type	Default	Description
RSize	int	DW + 2 +	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

2.3 Ports

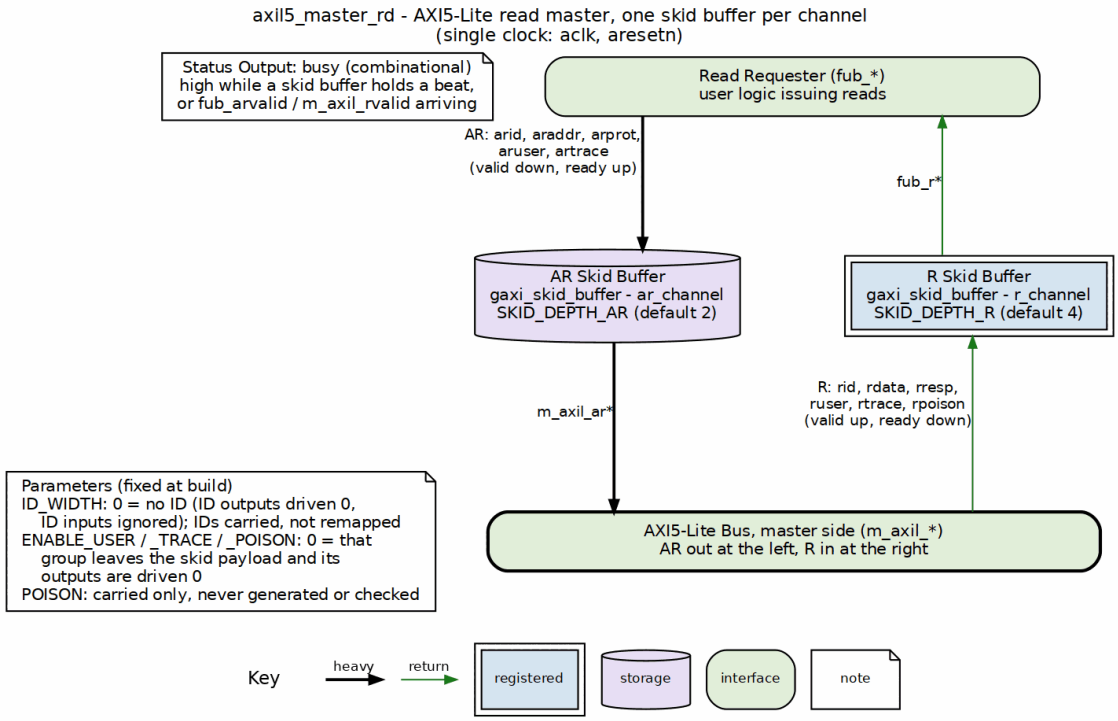
Your logic drives the `fub_*` side; the `m_axil_*` side faces the bus.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`fub_arid`	Input	`[IW-1:0]`	
`fub_araddr`	Input	`[AW-1:0]`	
`fub_arprot`	Input	`[2:0]`	
`fub_aruser`	Input	`[UW-1:0]`	
`fub_artrace`	Input	1	
`fub_arvalid`	Input	1	
`fub_arready`	Output	1	
`m_axil_arid`	Output	`[IW-1:0]`	
`m_axil_araddr`	Output	`[AW-1:0]`	
`m_axil_arprot`	Output	`[2:0]`	
`m_axil_aruser`	Output	`[UW-1:0]`	
`m_axil_artrace`	Output	1	
`m_axil_arvalid`	Output	1	
`m_axil_arready`	Input	1	
`m_axil_rid`	Input	`[IW-1:0]`	
`m_axil_rdata`	Input	`[DW-1:0]`	
`m_axil_rresp`	Input	`[1:0]`	
`m_axil_ruser`	Input	`[UW-1:0]`	
`m_axil_rtrace`	Input	1	
`m_axil_rpoison`	Input	`[PW-1:0]`	
`m_axil_rvalid`	Input	1	
`m_axil_rready`	Output	1	
`fub_rid`	Output	`[IW-1:0]`	

Port	Direction	Width	Description
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
busy	Output	1	

2.4 Functional Description

2.4.1 Block diagram



axil5_master_rd block diagram

The requester (`fub_*`) sits at the top and the AXI5-Lite bus (`m_axil_*`) is the bar at the bottom. AR runs down through its `gaxi_skid_buffer` to the bottom, and the R response climbs back up its own lane on the right through a second skid buffer, so every channel crosses exactly one buffer. This is the axil4 structure; AXI5-Lite only adds carried fields – the ID on every channel and the optional USER, TRACE and POISON groups, each dropped from the skid payload (outputs driven 0) when its `ENABLE_*` parameter is 0, as the Parameters legend states. `busy` is high while any skid buffer holds a beat or a valid is arriving (fub_arvalid, m_axil_rvalid), the same definition as every other AMBA base module.

2.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side

Group	Parameter	Signals on this module	Width
			modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

2.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

2.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given

leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the “tied LOW” of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

2.5 Timing Characteristics

2.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

2.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

2.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4)
) u_axil5_master_rd (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // fub_ar
    .fub_arid            (fub_arid),
    // ... `fub_araddr`, `fub_arprot`, `fub_aruser`, +3 more
    // m_axil_ar
    .m_axil_arid         (m_axil_arid),
    // ... `m_axil_araddr`, `m_axil_arprot`, `m_axil_aruser`, +3 more
    // m_axil_r
    .m_axil_rid          (m_axil_rid),
    // ... `m_axil_rdata`, `m_axil_rresp`, `m_axil_ruser`, +4 more
    // fub_r
    .fub_rid             (fub_rid),
    // ... `fub_rdata`, `fub_rresp`, `fub_ruser`, +4 more
    // status
    .busy                (busy)
);
```

2.7 Design Notes

No ID and all optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are sums of ID_WIDTH and conditional terms over the ENABLE_* parameters. With ID_WIDTH=0 (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts against both the AXI4-Lite RTL and axil5_master_rd configured that way.

Illegal widths do not elaborate. An initial begin : param_check block under `ifndef SYNTHESIS stops the build when AXIL_DATA_WIDTH is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or ID_WIDTH is outside 0..32 (1..32 on the slave-side modules). val/amba/test_axil_param_guard.py checks it.

Do not override a derived parameter. The *Size parameters and the AW/ DW/IW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The _mon/_monlite wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

2.8 Related Modules

- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd` – the AXI4-Lite counterpart
- AXI4-Lite modules

2.9 Testing

`val/amba/test_axil5_master_rd.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

2.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: storage on a back-pressured channel adds latency without changing function.
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI”, §8.3 “AXI4 interfaces (full and AXI-Lite)” – textbook treatment of the AXI-Lite channel set (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

2.11 Navigation

AXI5-Lite index | AMBA overview

3 AXIL5 Master Read, Clock-Gated

Module: `axil5_master_rd_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_cg`

3.1 Overview

The AXIL5 Master Read, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

3.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

3.3 Ports

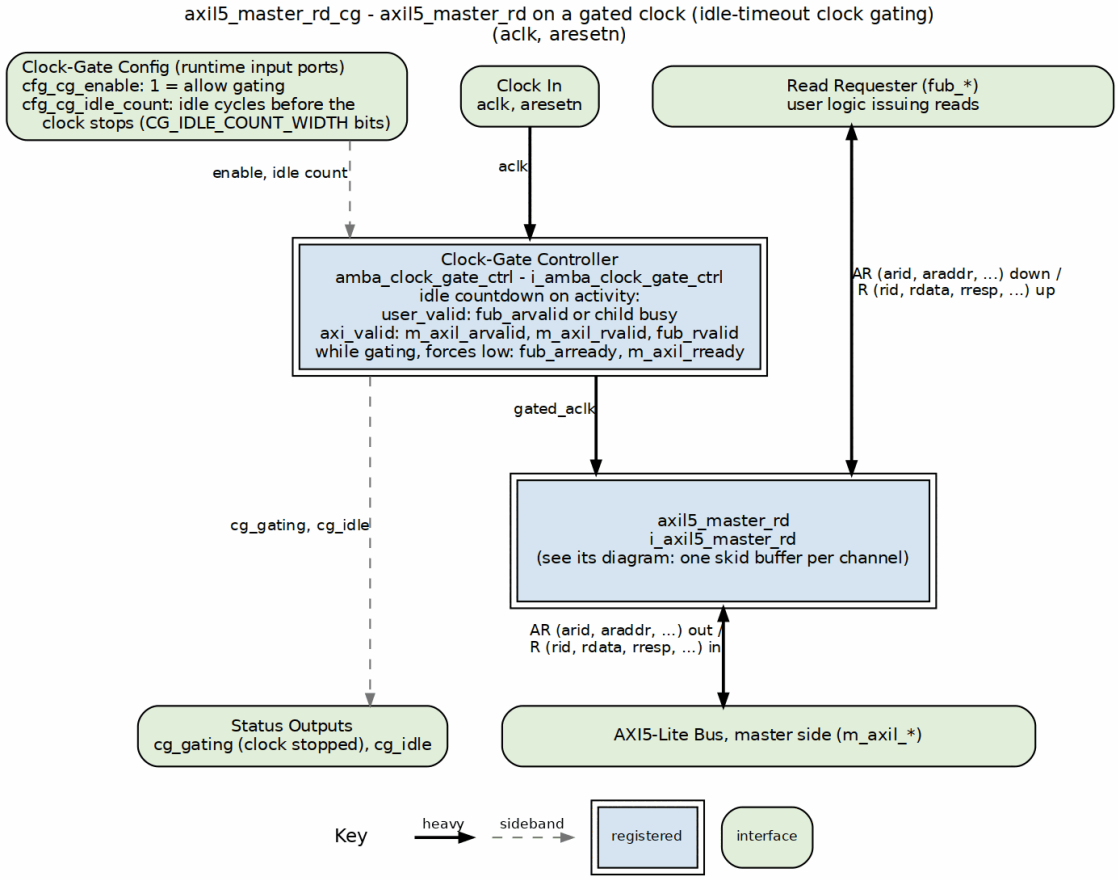
Your logic drives the `fub_*` side; the `m_axil_*` side faces the bus. The `cfg_cg_*` inputs run the clock gate.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cfg_cg_enable`	Input	1	
`cfg_cg_idle_count`	Input	`[CG_IDLE_COUNT_WIDTH-1:0]`	
`fub_arid`	Input	`[IW-1:0]`	
`fub_araddr`	Input	`[AW-1:0]`	
`fub_arprot`	Input	`[2:0]`	
`fub_aruser`	Input	`[UW-1:0]`	
`fub_artrace`	Input	1	
`fub_arvalid`	Input	1	
`fub_arready`	Output	1	
`m_axil_arid`	Output	`[IW-1:0]`	
`m_axil_araddr`	Output	`[AW-1:0]`	
`m_axil_arprot`	Output	`[2:0]`	
`m_axil_aruser`	Output	`[UW-1:0]`	
`m_axil_artrace`	Output	1	
`m_axil_arvalid`	Output	1	
`m_axil_arready`	Input	1	
`m_axil_rid`	Input	`[IW-1:0]`	
`m_axil_rdata`	Input	`[DW-1:0]`	
`m_axil_rresp`	Input	`[1:0]`	
`m_axil_ruser`	Input	`[UW-1:0]`	
`m_axil_rtrace`	Input	1	
`m_axil_rpoison`	Input	`[PW-1:0]`	

Port	Direction	Width	Description
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
fub_rid	Output	[IW-1:0]	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

3.4 Functional Description

3.4.1 Block diagram



axil5_master_rd_cg block diagram

`axil5_master_rd` is one child box (see its own diagram), clocked by `gated_aclk`. The heavy clock column runs from `aclk` through `amba_clock_gate_ctrl`, which counts down an idle window and stops the clock when nothing is active; its label lists the exact `user_valid` and `axi_valid` activity terms and the readies it forces low while gating, so no transfer is accepted on a stopped clock. `cg_gating` and `cg_idle` leave as status outputs.

3.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32;

Group	Parameter	Signals on this module	Width
	enable)		>= 1 on slave-side modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

3.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

3.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given

leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the “tied LOW” of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

3.5 Timing Characteristics

3.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

3.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

3.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_cg #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4),
    .CG_IDLE_COUNT_WIDTH (4)
) u_axil5_master_rd_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // configuration
    .cfg_cg_enable       (cfg_cg_enable),
    .cfg_cg_idle_count   (cfg_cg_idle_count),
    // fub_ar
    .fub_arid            (fub_arid),
    // ... `fub_araddr`, `fub_arprot`, `fub_aruser`, +3 more
    // m_axil_ar
    .m_axil_arid         (m_axil_arid),
    // ... `m_axil_araddr`, `m_axil_arprot`, `m_axil_aruser`, +3 more
    // m_axil_r
    .m_axil_rid          (m_axil_rid),
    // ... `m_axil_rdata`, `m_axil_rresp`, `m_axil_ruser`, +4 more
    // fub_r
    .fub_rid             (fub_rid),
    // ... `fub_rdata`, `fub_rresp`, `fub_ruser`, +4 more
    // status
    .cg_gating           (cg_gating),
    .cg_idle             (cg_idle)
);
```

3.7 Design Notes

No ID and all optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are sums of ID_WIDTH and conditional terms over the ENABLE_* parameters. With ID_WIDTH=0 (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts against both the AXI4-Lite RTL and axil5_master_rd configured that way.

Illegal widths do not elaborate. An initial begin : param_check block under `ifndef SYNTHESIS stops the build when AXIL_DATA_WIDTH is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or ID_WIDTH is outside 0..32 (1..32 on the slave-side modules). val/amba/test_axil_param_guard.py checks it.

Do not override a derived parameter. The *Size parameters and the AW/ DW/IW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The _mon/_monlite wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module's busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches amba_clock_gate_ctrl only. With it low the clock free-runs and the module behaves exactly like its base.

3.8 Related Modules

- axil5_master_rd
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

3.9 Testing

`val/amba/test_axil5_master_rd_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

3.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

3.11 Navigation

AXI5-Lite index | AMBA overview

4 AXIL5 Master Read Monitor

Module: `axil5_master_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon`

4.1 Overview

The AXIL5 Master Read Monitor provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled
- Integrated transaction monitor emitting monbus packets

4.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	`2`	
`SKID_DEPTH_R`	int	`4`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor, tie outputs
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions (reduced for AXIL)
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	
`ID_FILTER_ENABLE`	bit	`1'b0`	Per-instance ID-

Parameter	Type	Default	Description
			slice filter, inherited from the shared monitor core. Since amba #104 the wrapper feeds the AXI5-Lite ID, so the slice selects on AxID as on the AXI4 wrappers.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE=1. Compared against the AXI5-Lite AxID.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000A	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)

Parameter	Type	Default	Description
UNIT_ID		8'h01	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

4.3 Ports

Your logic drives the `fub_axil_*` side; the `m_axil_*` side faces the bus. The `cfg_*` and `monbus_*` ports belong to the monitor.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`fub_axil_arid`	Input	`[IW-1:0]`	
`fub_axil_araddr`	Input	`[AW-1:0]`	
`fub_axil_arprot`	Input	`[2:0]`	
`fub_axil_aruser`	Input	`[UW-1:0]`	
`fub_axil_artrace`	Input	1	
`fub_axil_arvalid`	Input	1	
`fub_axil_arready`	Output	1	
`fub_axil_rid`	Output	`[IW-1:0]`	
`fub_axil_rdata`	Output	`[DW-1:0]`	
`fub_axil_rresp`	Output	`[1:0]`	
`fub_axil_ruser`	Output	`[UW-1:0]`	
`fub_axil_rtrace`	Output	1	
`fub_axil_rpoison`	Output	`[PW-1:0]`	
`fub_axil_rvalid`	Output	1	
`fub_axil_rready`	Input	1	
`m_axil_arid`	Output	`[IW-1:0]`	
`m_axil_araddr`	Output	`[AW-1:0]`	
`m_axil_arprot`	Output	`[2:0]`	
`m_axil_aruser`	Output	`[UW-1:0]`	
`m_axil_artrace`	Output	1	
`m_axil_arvalid`	Output	1	
`m_axil_arready`	Input	1	

Port	Direction	Width	Description
m_axil_rid	Input	[IW-1:0]	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts

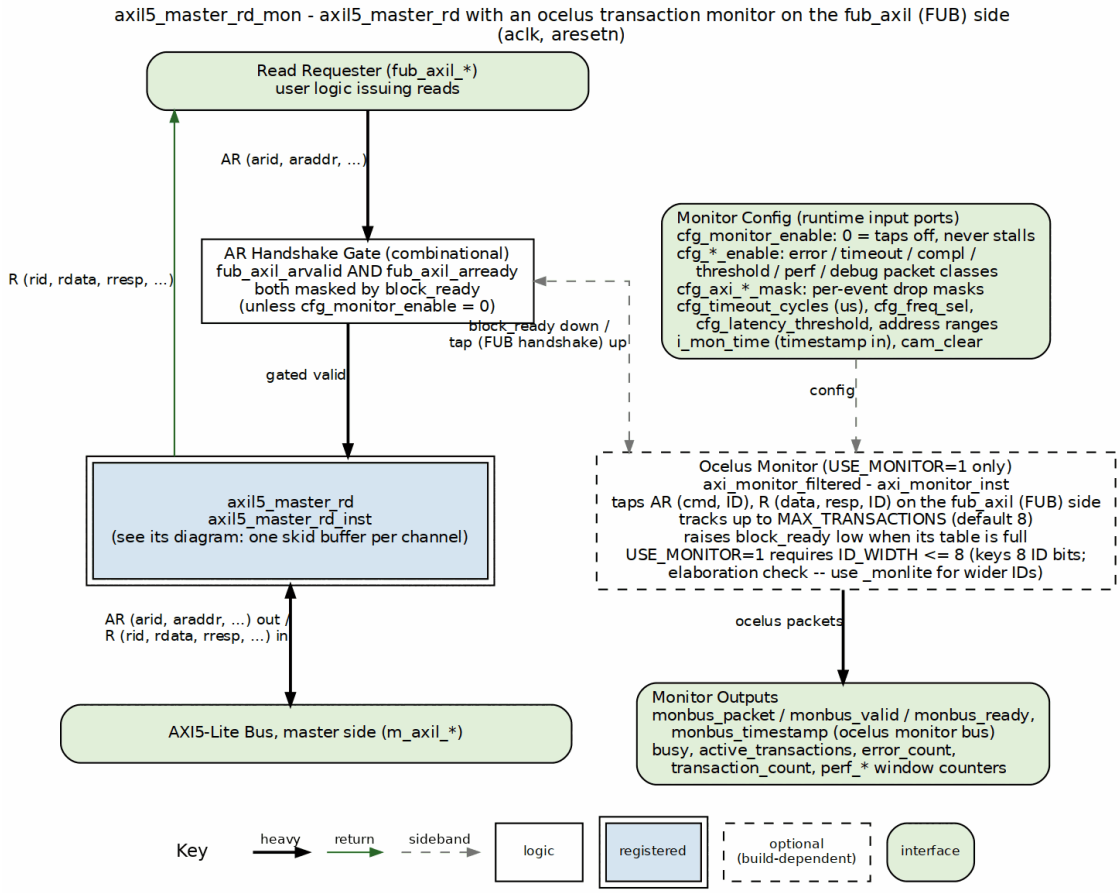
Port	Direction	Width	Description
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid

Port	Direction	Width	Description
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transacti	Output	[7:0]	Number of active

Port	Direction	Width	Description
ons			transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

4.4 Functional Description

4.4.1 Block diagram



axil5_master_rd_mon block diagram

`axil5_master_rd` is one child box (see its own diagram). The AR request first passes a handshake gate that masks valid and ready with `block_ready`, so a full monitor table stalls new requests instead of losing them. The optional ocelus monitor (`axi_monitor_filtered`, dashed because `USE_MONITOR=0` removes it) taps the FUB-side (`fub_axil`) handshake – the same point the handshake gate acts on, so there is no skid lag – including the transaction IDs, and emits monitor-bus packets as the heavy edge. With `USE_MONITOR=1` the build requires `ID_WIDTH <= 8`; the `_monlite` wrapper takes wider IDs.

4.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	ID_WIDTH (no enable)	AxID, BID, RID	ID_WIDTH (0..32; >= 1 on slave-side modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

4.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

4.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the

Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them "not present" on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

4.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

4.6 Timing Characteristics

4.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

4.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

4.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_mon #(
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ACLK_MHZ            (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .UNIT_ID             (8'h01),
    .AGENT_ID            (16'h000A),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_master_rd_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+31 more
    // m_axil_ar
    .m_axil_arid         (m_axil_arid),
    // ... `m_axil_araddr`, `m_axil_arprot`, `m_axil_aruser`, +3 more
```

```
    // m_axil_r
    .m_axil_rid          (m_axil_rid),
    // ... `m_axil_rdata`, `m_axil_rresp`, `m_axil_ruser`, +4 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

4.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

4.9 Testing

`val/amba/test_axil5_master_rd_mon.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

4.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

4.11 Navigation

AXI5-Lite index | AMBA overview

5 AXIL5 Master Read Monitor, Clock-Gated

Module: `axil5_master_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon_cg`

5.1 Overview

The AXIL5 Master Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

5.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axil5_master_rd_monlite_cg`: the same gating logic around `axil5_master_rd_monlite` instead of `axil5_master_rd_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	`2`	
`SKID_DEPTH_R`	int	`4`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor in inner mon; outputs tied
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`ADDR_FILTER_ENABLE`	bit	`1'b0`	0 = address filter inert
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions (reduced for

Parameter	Type	Default	Description
			AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0

Parameter	Type	Default	Description
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000A	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

5.3 Ports

Your logic drives the `fub_axil_*` side; the `m_axil_*` side faces the bus. The `cfg_*` and `monbus_*` ports belong to the monitor, and `cfg_cg_*` runs the clock gate.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`fub_axil_arid`	Input	`[IW-1:0]`	
`fub_axil_araddr`	Input	`[AW-1:0]`	
`fub_axil_arprot`	Input	`[2:0]`	
`fub_axil_aruser`	Input	`[UW-1:0]`	
`fub_axil_artrace`	Input	1	
`fub_axil_arvalid`	Input	1	
`fub_axil_arready`	Output	1	
`fub_axil_rid`	Output	`[IW-1:0]`	
`fub_axil_rdata`	Output	`[DW-1:0]`	
`fub_axil_rresp`	Output	`[1:0]`	
`fub_axil_ruser`	Output	`[UW-1:0]`	
`fub_axil_rtrace`	Output	1	
`fub_axil_rpoison`	Output	`[PW-1:0]`	
`fub_axil_rvalid`	Output	1	
`fub_axil_rready`	Input	1	
`m_axil_arid`	Output	`[IW-1:0]`	
`m_axil_araddr`	Output	`[AW-1:0]`	
`m_axil_arprot`	Output	`[2:0]`	
`m_axil_aruser`	Output	`[UW-1:0]`	
`m_axil_artrace`	Output	1	
`m_axil_arvalid`	Output	1	
`m_axil_arready`	Input	1	

Port	Direction	Width	Description
m_axil_rid	Input	[IW-1:0]	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts

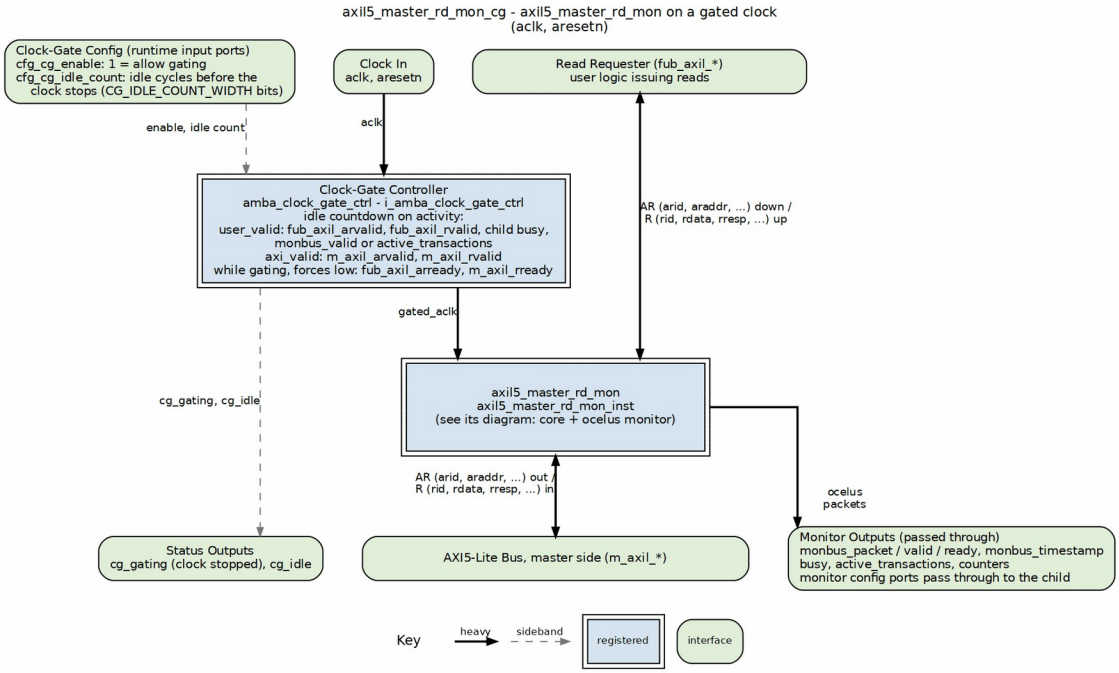
Port	Direction	Width	Description
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	

Port	Direction	Width	Description
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs

Port	Direction	Width	Description
			with `cfg_addr_range_low`.
`busy`	Output	1	
`active_transactions`	Output	`[7:0]`	Number of active transactions
`error_count`	Output	`[15:0]`	Total error count
`transaction_count`	Output	`[31:0]`	Total transaction count
`cg_gating`	Output	1	Gated clock is stopped
`cg_idle`	Output	1	No activity observed
`cfg_conflict_error`	Output	1	Configuration conflict detected
`cfg_start_event_sel`	Input	`[2:0]`	
`cfg_end_event_sel`	Input	`[2:0]`	
`cfg_start_trigger`	Input	1	
`cfg_end_trigger`	Input	1	
`cfg_window_force_close`	Input	1	
`window_active`	Output	1	
`window_cycles`	Output	`[31:0]`	
`perf_prod_cycles`	Output	`[31:0]`	
`perf_bp_cycles`	Output	`[31:0]`	
`perf_starv_cycles`	Output	`[31:0]`	
`perf_idle_cycles`	Output	`[31:0]`	
`perf_beat_count`	Output	`[31:0]`	
`perf_byte_count`	Output	`[63:0]`	
`perf_burst_count`	Output	`[31:0]`	

5.4 Functional Description

5.4.1 Block diagram



axil5_master_rd_mon_cg block diagram

`axil5_master_rd_mon` (core plus ocelus monitor) is one child box (see its own diagram), clocked by `gated_aclk` from `amba_clock_gate_ctrl`. The controller's label lists the activity terms, which add `monbus_valid` and `active_transactions` so the clock keeps running while the monitor still has packets or open transactions. Monitor-bus packets and the monitor configuration pass straight through to the child.

5.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`

Group	Parameter	Signals on this module	Width
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

5.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

5.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

5.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

5.6 Timing Characteristics

5.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

5.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

5.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_mon_cg #(
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ACLK_MHZ            (100),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .UNIT_ID             (8'h01),
    .AGENT_ID            (16'h000A),
    .MAX_TRANSACTIONS    (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .CG_IDLE_COUNT_WIDTH (4),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_master_rd_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+32 more
    // m_axil_ar
    .m_axil_arid         (m_axil_arid),
    // ... `m_axil_araddr`, `m_axil_arprot`, `m_axil_aruser`, +3 more
    // m_axil_r
    .m_axil_rid          (m_axil_rid),
```

```
    // ... `m_axil_rdata`, `m_axil_rresp`, `m_axil_ruser`, +4 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

5.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

5.9 Testing

`val/amba/test_axil5_master_rd_mon_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

5.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- READY held LOW while gated is legal: a receiver may keep READY LOW until it chooses to accept (IHI 0022H §A3.2.1; IHI 0022L §A2.3.1). No paper (standard technique).
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

5.11 Navigation

AXI5-Lite index | AMBA overview

6 AXIL5 Master Write

Module: `axil5_master_wr.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr`

6.1 Overview

The AXIL5 Master Write provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled

6.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ?	

Parameter	Type	Default	Description
		`(DW / 64) : 1`	
`AWSize`	int	`AW + 3 +`	
`WSize`	int	`DW + SW +`	
`BSize`	int	`2 +`	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

6.3 Ports

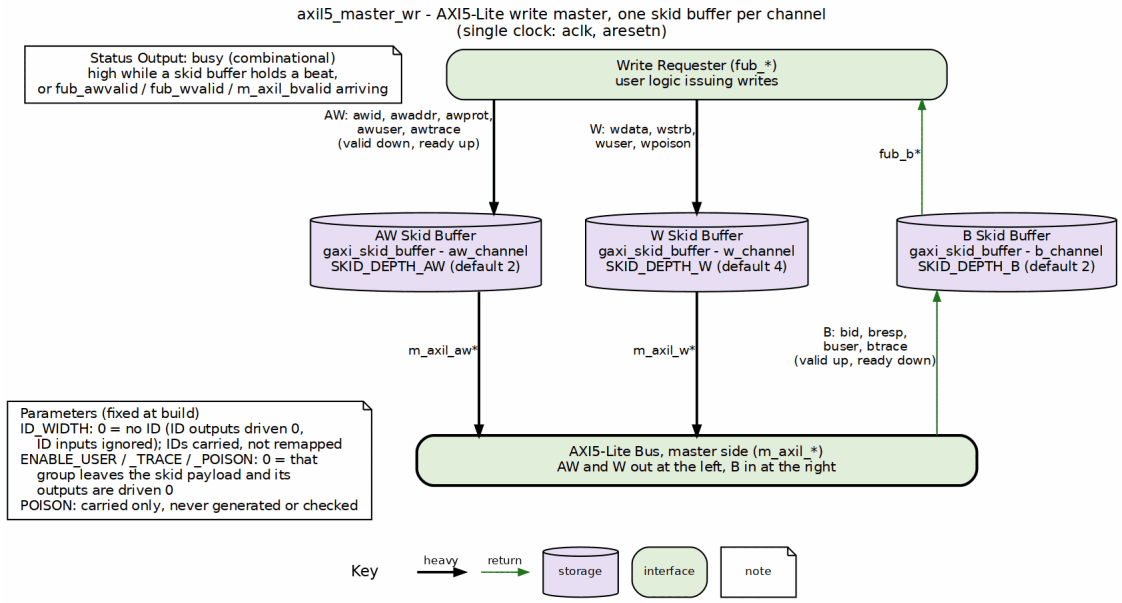
Your logic drives the `fub_*` side; the `m_axil_*` side faces the bus.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`fub_awid`	Input	`[IW-1:0]`	
`fub_awaddr`	Input	`[AW-1:0]`	
`fub_awprot`	Input	`[2:0]`	
`fub_awuser`	Input	`[UW-1:0]`	
`fub_awtrace`	Input	1	
`fub_awvalid`	Input	1	
`fub_awready`	Output	1	
`m_axil_awid`	Output	`[IW-1:0]`	
`m_axil_awaddr`	Output	`[AW-1:0]`	
`m_axil_awprot`	Output	`[2:0]`	
`m_axil_awuser`	Output	`[UW-1:0]`	
`m_axil_awtrace`	Output	1	
`m_axil_awvalid`	Output	1	
`m_axil_awready`	Input	1	
`fub_wdata`	Input	`[DW-1:0]`	
`fub_wstrb`	Input	`[SW-1:0]`	
`fub_wuser`	Input	`[UW-1:0]`	
`fub_wpoison`	Input	`[PW-1:0]`	
`fub_wvalid`	Input	1	
`fub_wready`	Output	1	
`m_axil_wdata`	Output	`[DW-1:0]`	
`m_axil_wstrb`	Output	`[SW-1:0]`	
`m_axil_wuser`	Output	`[UW-1:0]`	

Port	Direction	Width	Description
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bid	Input	[IW-1:0]	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bid	Output	[IW-1:0]	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bvalid	Output	1	
fub_bready	Input	1	
busy	Output	1	

6.4 Functional Description

6.4.1 Block diagram



axil5_master_wr block diagram

The requester (`fub_*`) sits at the top and the AXI5-Lite bus (`m_axil_*`) is the bar at the bottom. AW and W run down through their own `gaxi_skid_buffers`, and the B response climbs back up its own lane on the right through a third, so every channel crosses exactly one buffer. This is the axil4 structure; AXI5-Lite only adds carried fields – the ID on every channel and the optional USER, TRACE and POISON groups, each dropped from the skid payload (outputs driven 0) when its `ENABLE_*` parameter is 0, as the Parameters legend states. `busy` is high while any skid buffer holds a beat or a valid is arriving (fub_awvalid, fub_wvalid, m_axil_bvalid), the same definition as every other AMBA base module.

6.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)
USER	`ENABLE_USER`	`AxUSER`, `WUSER`,	`USER_WIDTH`

Group	Parameter	Signals on this module	Width
		BUSER, RUSER	
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

6.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

6.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

6.5 Timing Characteristics

6.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

6.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

6.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (4),
    .SKID_DEPTH_B        (2)
) u_axil5_master_wr (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // fub_aw
    .fub_awid            (fub_awid),
    // ... `fub_awaddr`, `fub_awprot`, `fub_awuser`, +3 more
    // m_axil_aw
    .m_axil_awid         (m_axil_awid),
    // ... `m_axil_awaddr`, `m_axil_awprot`, `m_axil_awuser`, +3 more
    // fub_w
    .fub_wdata           (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // m_axil_w
    .m_axil_wdata        (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bid          (m_axil_bid),
    // ... `m_axil_bresp`, `m_axil_buser`, `m_axil_btrace`, +2 more
    // fub_b
    .fub_bid             (fub_bid),
    // ... `fub_bresp`, `fub_buser`, `fub_btrace`, +2 more
    // status
    .busy                (busy)
);
```

6.7 Design Notes

No ID and all optional groups off is AXI4-Lite. `ARSize`/`AWSize`/`WSize`/`RSize`/ `BSize` are sums of `ID_WIDTH` and conditional terms over the `ENABLE_*` parameters. With `ID_WIDTH=0` (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what `val/amba/test_axil5_axil4_equivalence.py` asserts against both the AXI4-Lite RTL and `axil5_master_rd` configured that way.

Illegal widths do not elaborate. An `initial begin : param_check` block under `` `ifndef SYNTHESIS `` stops the build when `AXIL_DATA_WIDTH` is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or `ID_WIDTH` is outside 0..32 (1..32 on the slave-side modules). `val/amba/test_axil_param_guard.py` checks it.

Do not override a derived parameter. The `*Size` parameters and the `AW`/ `DW`/`IW` aliases are declared as `parameter` so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The `_mon`/`_monlite` wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

6.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_wr` – the AXI4-Lite counterpart
- AXI4-Lite modules

6.9 Testing

`val/amba/test_axil5_master_wr.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

6.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: storage on a back-pressured channel adds latency without changing function.
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI”, §8.3 “AXI4 interfaces (full and AXI-Lite)” – textbook treatment of the AXI-Lite channel set (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

6.11 Navigation

AXI5-Lite index | AMBA overview

7 AXIL5 Master Write, Clock-Gated

Module: `axil5_master_wr_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_cg`

7.1 Overview

The AXIL5 Master Write, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

7.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0

Parameter	Type	Default	Description
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

7.3 Ports

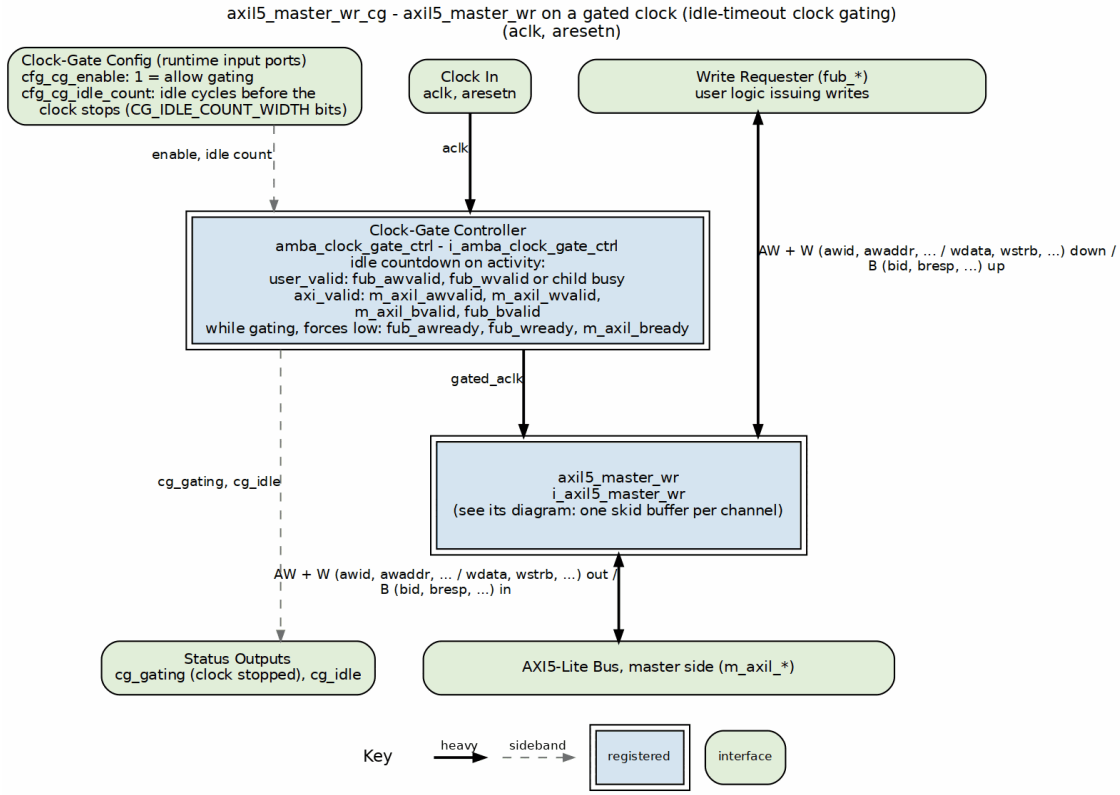
Your logic drives the `fub_*` side; the `m_axil_*` side faces the bus. The `cfg_cg_*` inputs run the clock gate.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cfg_cg_enable`	Input	1	
`cfg_cg_idle_count`	Input	`[CG_IDLE_COUNT_WIDTH-1:0]`	
`fub_awid`	Input	`[IW-1:0]`	
`fub_awaddr`	Input	`[AW-1:0]`	
`fub_awprot`	Input	`[2:0]`	
`fub_awuser`	Input	`[UW-1:0]`	
`fub_awtrace`	Input	1	
`fub_awvalid`	Input	1	
`fub_awready`	Output	1	
`m_axil_awid`	Output	`[IW-1:0]`	
`m_axil_awaddr`	Output	`[AW-1:0]`	
`m_axil_awprot`	Output	`[2:0]`	
`m_axil_awuser`	Output	`[UW-1:0]`	
`m_axil_awtrace`	Output	1	
`m_axil_awvalid`	Output	1	
`m_axil_awready`	Input	1	
`fub_wdata`	Input	`[DW-1:0]`	
`fub_wstrb`	Input	`[SW-1:0]`	
`fub_wuser`	Input	`[UW-1:0]`	
`fub_wpoison`	Input	`[PW-1:0]`	
`fub_wvalid`	Input	1	
`fub_wready`	Output	1	
`m_axil_wdata`	Output	`[DW-1:0]`	

Port	Direction	Width	Description
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bid	Input	[IW-1:0]	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bid	Output	[IW-1:0]	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bvalid	Output	1	
fub_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

7.4 Functional Description

7.4.1 Block diagram



axil5_master_wr_cg block diagram

`axil5_master_wr` is one child box (see its own diagram), clocked by `gated_aclk`. The heavy clock column runs from `aclk` through `amba_clock_gate_ctrl`, which counts down an idle window and stops the clock when nothing is active; its label lists the exact `user_valid` and `axi_valid` activity terms and the readies it forces low while gating, so no transfer is accepted on a stopped clock. `cg_gating` and `cg_idle` leave as status outputs.

7.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

7.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

7.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

7.5 Timing Characteristics

7.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

7.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

7.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_cg #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (4),
    .SKID_DEPTH_B        (2),
    .CG_IDLE_COUNT_WIDTH (4)
) u_axil5_master_wr_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // configuration
    .cfg_cg_enable       (cfg_cg_enable),
    .cfg_cg_idle_count   (cfg_cg_idle_count),
    // fub_aw
    .fub_awid            (fub_awid),
    // ... `fub_awaddr`, `fub_awprot`, `fub_awuser`, +3 more
    // m_axil_aw
    .m_axil_awid         (m_axil_awid),
    // ... `m_axil_awaddr`, `m_axil_awprot`, `m_axil_awuser`, +3 more
    // fub_w
    .fub_wdata           (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // m_axil_w
    .m_axil_wdata        (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bid          (m_axil_bid),
    // ... `m_axil_bresp`, `m_axil_buser`, `m_axil_btrace`, +2 more
    // fub_b
    .fub_bid             (fub_bid),
    // ... `fub_bresp`, `fub_buser`, `fub_btrace`, +2 more
    // status
    .cg_gating           (cg_gating),
    .cg_idle             (cg_idle)
);
```

7.7 Design Notes

No ID and all optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are sums of ID_WIDTH and conditional terms over the ENABLE_* parameters. With ID_WIDTH=0 (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts against both the AXI4-Lite RTL and axil5_master_rd configured that way.

Illegal widths do not elaborate. An initial begin : param_check block under `ifndef SYNTHESIS stops the build when AXIL_DATA_WIDTH is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or ID_WIDTH is outside 0..32 (1..32 on the slave-side modules). val/amba/test_axil_param_guard.py checks it.

Do not override a derived parameter. The *Size parameters and the AW/ DW/IW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The _mon/_monlite wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module's busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches amba_clock_gate_ctrl only. With it low the clock free-runs and the module behaves exactly like its base.

7.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_mon
- `axil4_master_wr_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

7.9 Testing

`val/amba/test_axil5_master_wr_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

7.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

7.11 Navigation

AXI5-Lite index | AMBA overview

8 AXIL5 Master Write Monitor

Module: `axil5_master_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon`

8.1 Overview

The AXIL5 Master Write Monitor provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled
- Integrated transaction monitor emitting monbus packets

8.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor, tie outputs
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions (reduced for AXIL)
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	

Parameter	Type	Default	Description
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Since amba #104 the wrapper feeds the AXI5-Lite ID, so the slice selects on AxID as on the AXI4 wrappers.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE=1. Compared against the AXI5-Lite AxID.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000B	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-

Parameter	Type	Default	Description
			bit Agent ID for monitor packets)
`UNIT_ID`		`8'h01`	Unit identifier emitted in the `unit_id` field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

8.3 Ports

Your logic drives the `fub_axil_*` side; the `m_axil_*` side faces the bus. The `cfg_*` and `monbus_*` ports belong to the monitor.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`fub_axil_awid`	Input	`[IW-1:0]`	
`fub_axil_awaddr`	Input	`[AW-1:0]`	
`fub_axil_awprot`	Input	`[2:0]`	
`fub_axil_awuser`	Input	`[UW-1:0]`	
`fub_axil_awtrace`	Input	1	
`fub_axil_awvalid`	Input	1	
`fub_axil_awready`	Output	1	
`fub_axil_wdata`	Input	`[DW-1:0]`	
`fub_axil_wstrb`	Input	`[DW/8-1:0]`	
`fub_axil_wuser`	Input	`[UW-1:0]`	
`fub_axil_wpoison`	Input	`[PW-1:0]`	
`fub_axil_wvalid`	Input	1	
`fub_axil_wready`	Output	1	
`fub_axil_bid`	Output	`[IW-1:0]`	
`fub_axil_bresp`	Output	`[1:0]`	
`fub_axil_buser`	Output	`[UW-1:0]`	
`fub_axil_btrace`	Output	1	
`fub_axil_bvalid`	Output	1	
`fub_axil_bready`	Input	1	
`m_axil_awid`	Output	`[IW-1:0]`	
`m_axil_awaddr`	Output	`[AW-1:0]`	
`m_axil_awprot`	Output	`[2:0]`	

Port	Direction	Width	Description
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bid	Input	[IW-1:0]	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets

Port	Direction	Width	Description
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	

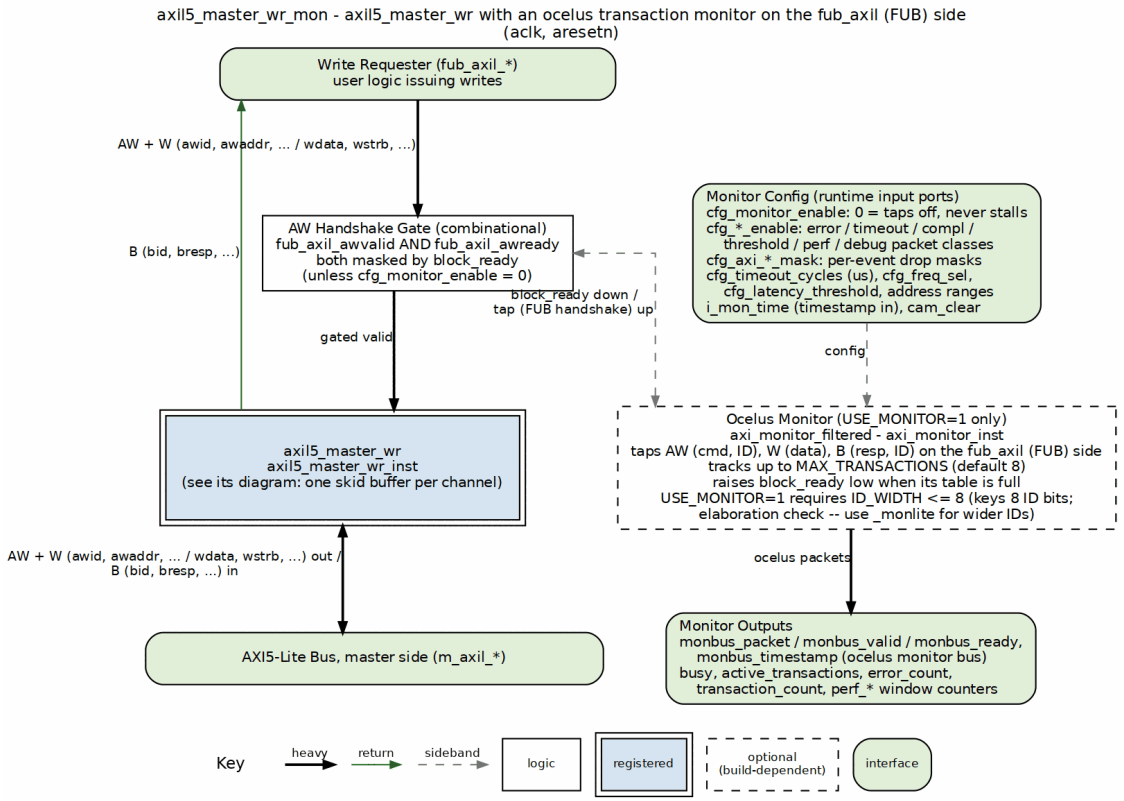
Port	Direction	Width	Description
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES >

Port	Direction	Width	Description
			0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	

Port	Direction	Width	Description
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

8.4 Functional Description

8.4.1 Block diagram



axil5_master_wr_mon block diagram

`axil5_master_wr` is one child box (see its own diagram). The AW request first passes a handshake gate that masks valid and ready with `block_ready`, so a full monitor table stalls new requests instead of losing them. The optional ocelus monitor (`axi_monitor_filtered`, dashed because `USE_MONITOR=0` removes it) taps the FUB-side (`fub_axil`) handshake – the same point the handshake gate acts on, so there is no skid lag – including the transaction IDs, and emits monitor-bus packets as the heavy edge. With `USE_MONITOR=1` the build requires `ID_WIDTH <= 8`; the `_monlite` wrapper takes wider IDs.

8.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32;

Group	Parameter	Signals on this module	Width
	enable)		>= 1 on slave-side modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

8.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

8.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given

leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the “tied LOW” of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

8.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

8.6 Timing Characteristics

8.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

8.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

8.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_mon #(
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (2),
    .SKID_DEPTH_B        (2),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ACLK_MHZ            (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .UNIT_ID             (8'h01),
    .AGENT_ID            (16'h000B),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_master_wr_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+35 more
    // m_axil_aw
    .m_axil_awid         (m_axil_awid),
```

```
    // ... `m_axil_awaddr`, `m_axil_awprot`, `m_axil_awuser`, +3 more
    // m_axil_w
    .m_axil_wdata        (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bid          (m_axil_bid),
    // ... `m_axil_bresp`, `m_axil_buser`, `m_axil_btrace`, +2 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

8.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_master_wr_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

8.9 Testing

`val/amba/test_axil5_master_wr_mon.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

8.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

8.11 Navigation

AXI5-Lite index | AMBA overview

9 AXIL5 Master Write Monitor, Clock-Gated

Module: `axil5_master_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon_cg`

9.1 Overview

The AXIL5 Master Write Monitor, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with `ID_WIDTH=0` and all groups disabled
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

9.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axil5_master_wr_monlite_cg`: the same gating logic around `axil5_master_wr_monlite` instead of `axil5_master_wr_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor in inner mon; outputs tied
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`ADDR_FILTER_ENABLE`	bit	`1'b0`	0 = address filter inert
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions

Parameter	Type	Default	Description
			(reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 0..32; 0 = no ID (Manager side, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder

Parameter	Type	Default	Description
			when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000B	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

9.3 Ports

Your logic drives the `fub_axil_*` side; the `m_axil_*` side faces the bus. The `cfg_*` and `monbus_*` ports belong to the monitor, and `cfg_cg_*` runs the clock gate.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`fub_axil_awid`	Input	`[IW-1:0]`	
`fub_axil_awaddr`	Input	`[AW-1:0]`	
`fub_axil_awprot`	Input	`[2:0]`	
`fub_axil_awuser`	Input	`[UW-1:0]`	
`fub_axil_awtrace`	Input	1	
`fub_axil_awvalid`	Input	1	
`fub_axil_awready`	Output	1	
`fub_axil_wdata`	Input	`[DW-1:0]`	
`fub_axil_wstrb`	Input	`[DW/8-1:0]`	
`fub_axil_wuser`	Input	`[UW-1:0]`	
`fub_axil_wpoison`	Input	`[PW-1:0]`	
`fub_axil_wvalid`	Input	1	
`fub_axil_wready`	Output	1	
`fub_axil_bid`	Output	`[IW-1:0]`	
`fub_axil_bresp`	Output	`[1:0]`	
`fub_axil_buser`	Output	`[UW-1:0]`	
`fub_axil_btrace`	Output	1	
`fub_axil_bvalid`	Output	1	
`fub_axil_bready`	Input	1	
`m_axil_awid`	Output	`[IW-1:0]`	
`m_axil_awaddr`	Output	`[AW-1:0]`	
`m_axil_awprot`	Output	`[2:0]`	

Port	Direction	Width	Description
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bid	Input	[IW-1:0]	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets

Port	Direction	Width	Description
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating

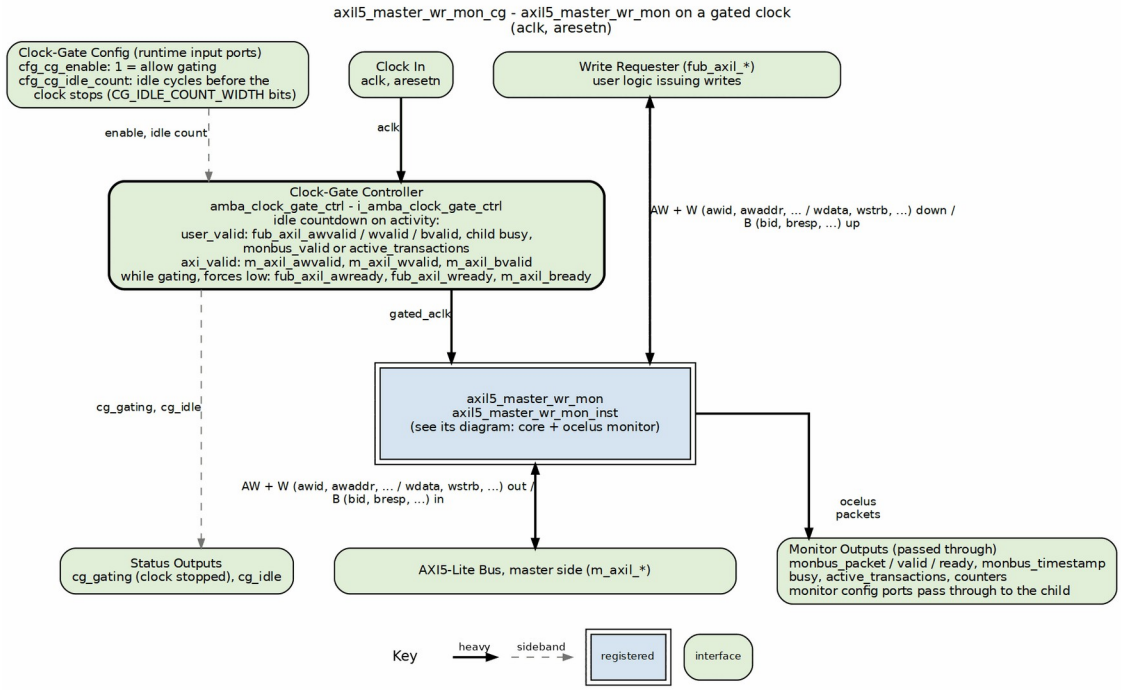
Port	Direction	Width	Description
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-

Port	Direction	Width	Description
			range comparator; only present when `N_ADDR_RANGES > 0`.
`cfg_addr_range_high`	Input	`[AW-1:0]` x `N_ADDR_RANGES`	High bound of each address-range comparator; pairs with `cfg_addr_range_low`.
`busy`	Output	1	
`active_transactions`	Output	`[7:0]`	Number of active transactions
`error_count`	Output	`[15:0]`	Total error count
`transaction_count`	Output	`[31:0]`	Total transaction count
`cg_gating`	Output	1	Gated clock is stopped
`cg_idle`	Output	1	No activity observed
`cfg_conflict_error`	Output	1	Configuration conflict detected
`cfg_start_event_sel`	Input	`[2:0]`	
`cfg_end_event_sel`	Input	`[2:0]`	
`cfg_start_trigger`	Input	1	
`cfg_end_trigger`	Input	1	
`cfg_window_force_close`	Input	1	
`window_active`	Output	1	
`window_cycles`	Output	`[31:0]`	

Port	Direction	Width	Description
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

9.4 Functional Description

9.4.1 Block diagram



axil5_master_wr_mon_cg block diagram

`axil5_master_wr_mon` (core plus ocelus monitor) is one child box (see its own diagram), clocked by `gated_aclk` from `amba_clock_gate_ctrl`. The controller's label lists the activity terms, which add `monbus_valid` and `active_transactions` so the clock keeps running while the monitor still has packets or open transactions. Monitor-bus packets and the monitor configuration pass straight through to the child.

9.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`

Group	Parameter	Signals on this module	Width
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

9.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

9.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

9.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

9.6 Timing Characteristics

9.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

9.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

9.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_mon_cg #(
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (2),
    .SKID_DEPTH_B        (2),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ACLK_MHZ            (100),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .UNIT_ID             (8'h01),
    .AGENT_ID            (16'h000B),
    .MAX_TRANSACTIONS    (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .CG_IDLE_COUNT_WIDTH (4),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_master_wr_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+36 more
    // m_axil_aw
    .m_axil_awid         (m_axil_awid),
    // ... `m_axil_awaddr`, `m_axil_awprot`, `m_axil_awuser`, +3 more
    // m_axil_w
```

```
    .m_axil_wdata        (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bid          (m_axil_bid),
    // ... `m_axil_bresp`, `m_axil_buser`, `m_axil_btrace`, +2 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

9.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_master_wr_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

9.9 Testing

`val/amba/test_axil5_master_wr_mon_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

9.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- READY held LOW while gated is legal: a receiver may keep READY LOW until it chooses to accept (IHI 0022H §A3.2.1; IHI 0022L §A2.3.1). No paper (standard technique).
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

9.11 Navigation

AXI5-Lite index | AMBA overview

10 AXIL5 Slave Read

Module: `axil5_slave_rd.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd`

10.1 Overview

The AXIL5 Slave Read provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)

10.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
ARSize	int	AW + 3 +	

Parameter	Type	Default	Description
RSize	int	DW + 2 +	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

10.3 Ports

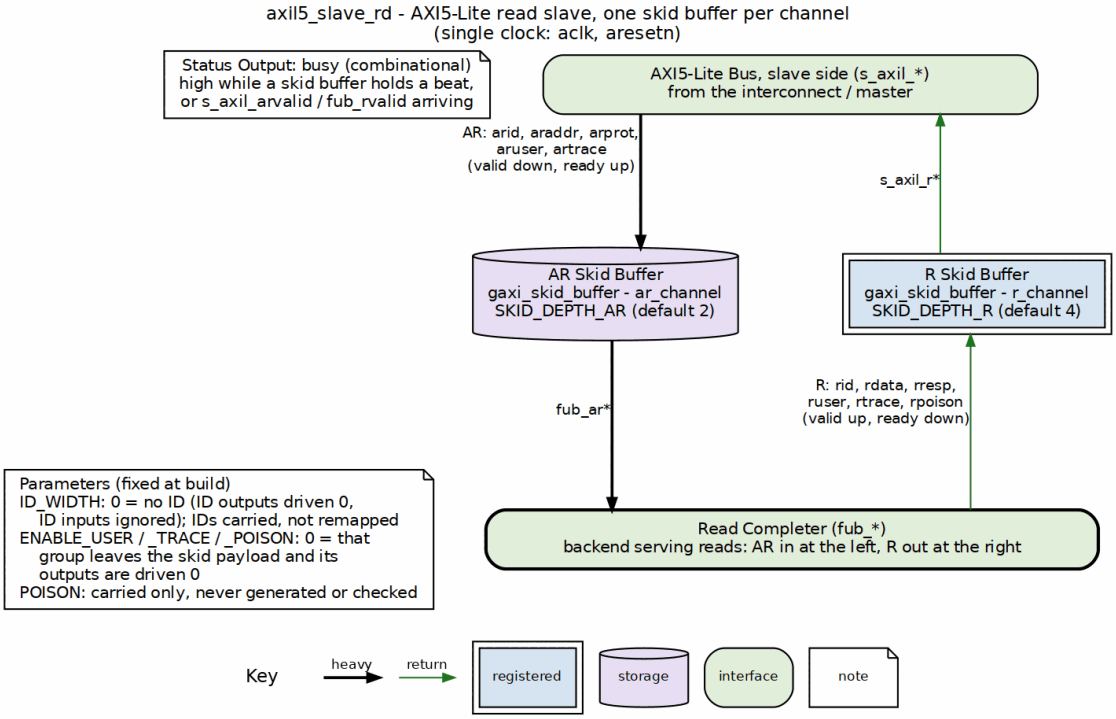
The `s_axil_*` side faces the bus; your backend drives the `fub_*` side.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`s_axil_arid`	Input	`[IW-1:0]`	
`s_axil_araddr`	Input	`[AW-1:0]`	
`s_axil_arprot`	Input	`[2:0]`	
`s_axil_aruser`	Input	`[UW-1:0]`	
`s_axil_artrace`	Input	1	
`s_axil_arvalid`	Input	1	
`s_axil_arready`	Output	1	
`fub_arid`	Output	`[IW-1:0]`	
`fub_araddr`	Output	`[AW-1:0]`	
`fub_arprot`	Output	`[2:0]`	
`fub_aruser`	Output	`[UW-1:0]`	
`fub_artrace`	Output	1	
`fub_arvalid`	Output	1	
`fub_arready`	Input	1	
`fub_rid`	Input	`[IW-1:0]`	
`fub_rdata`	Input	`[DW-1:0]`	
`fub_rresp`	Input	`[1:0]`	
`fub_ruser`	Input	`[UW-1:0]`	
`fub_rtrace`	Input	1	
`fub_rpoison`	Input	`[PW-1:0]`	
`fub_rvalid`	Input	1	
`fub_rready`	Output	1	
`s_axil_rid`	Output	`[IW-1:0]`	
`s_axil_rdata`	Output	`[DW-1:0]`	

Port	Direction	Width	Description
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
busy	Output	1	

10.4 Functional Description

10.4.1 Block diagram



axil5_slave_rd block diagram

The AXI5-Lite bus (`s_axil_*`) sits at the top and the completer (`fub_*`) is the bar at the bottom. AR runs down through its `gaxi_skid_buffer` to the bottom, and the R response climbs back up its own lane on the right through a second skid buffer, so every channel crosses exactly one buffer. This is the axil4 structure; AXI5-Lite only adds carried fields – the ID on every channel and the optional USER, TRACE and POISON groups, each dropped from the skid payload (outputs driven 0) when its `ENABLE_*` parameter is 0, as the Parameters legend states. `busy` is high while any skid buffer holds a beat or a valid is arriving (s_axil_arvalid, fub_rvalid), the same definition as every other AMBA base module.

10.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side

Group	Parameter	Signals on this module	Width
			modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

10.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

10.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given

leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the “tied LOW” of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

10.5 Timing Characteristics

10.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

10.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

10.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4)
) u_axil5_slave_rd (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // s_axil_ar
    .s_axil_arid         (s_axil_arid),
    // ... `s_axil_araddr`, `s_axil_arprot`, `s_axil_aruser`, +3 more
    // fub_ar
    .fub_arid            (fub_arid),
    // ... `fub_araddr`, `fub_arprot`, `fub_aruser`, +3 more
    // fub_r
    .fub_rid             (fub_rid),
    // ... `fub_rdata`, `fub_rresp`, `fub_ruser`, +4 more
    // s_axil_r
    .s_axil_rid          (s_axil_rid),
    // ... `s_axil_rdata`, `s_axil_rresp`, `s_axil_ruser`, +4 more
    // status
    .busy                (busy)
);
```

10.7 Design Notes

No ID and all optional groups off is AXI4-Lite. `ARSize`/`AWSize`/`WSize`/`RSize`/ `BSize` are sums of `ID_WIDTH` and conditional terms over the `ENABLE_*` parameters. With `ID_WIDTH=0` (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what `val/amba/test_axil5_axil4_equivalence.py` asserts against both the AXI4-Lite RTL and `axil5_master_rd` configured that way.

Illegal widths do not elaborate. An `initial begin : param_check` block under `` `ifndef SYNTHESIS `` stops the build when `AXIL_DATA_WIDTH` is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or `ID_WIDTH` is outside 0..32 (1..32 on the slave-side modules). `val/amba/test_axil_param_guard.py` checks it.

Do not override a derived parameter. The `*Size` parameters and the `AW`/ `DW`/`IW` aliases are declared as `parameter` so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The `_mon`/`_monlite` wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

10.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd` – the AXI4-Lite counterpart
- AXI4-Lite modules

10.9 Testing

`val/amba/test_axil5_slave_rd.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

10.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: storage on a back-pressured channel adds latency without changing function.
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI”, §8.3 “AXI4 interfaces (full and AXI-Lite)” – textbook treatment of the AXI-Lite channel set (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

10.11 Navigation

AXI5-Lite index | AMBA overview

11 AXIL5 Slave Read, Clock-Gated

Module: `axil5_slave_rd_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_cg`

11.1 Overview

The AXIL5 Slave Read, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)
- One `amba_clock_gate_ctrl` gating the whole inner module

11.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

11.3 Ports

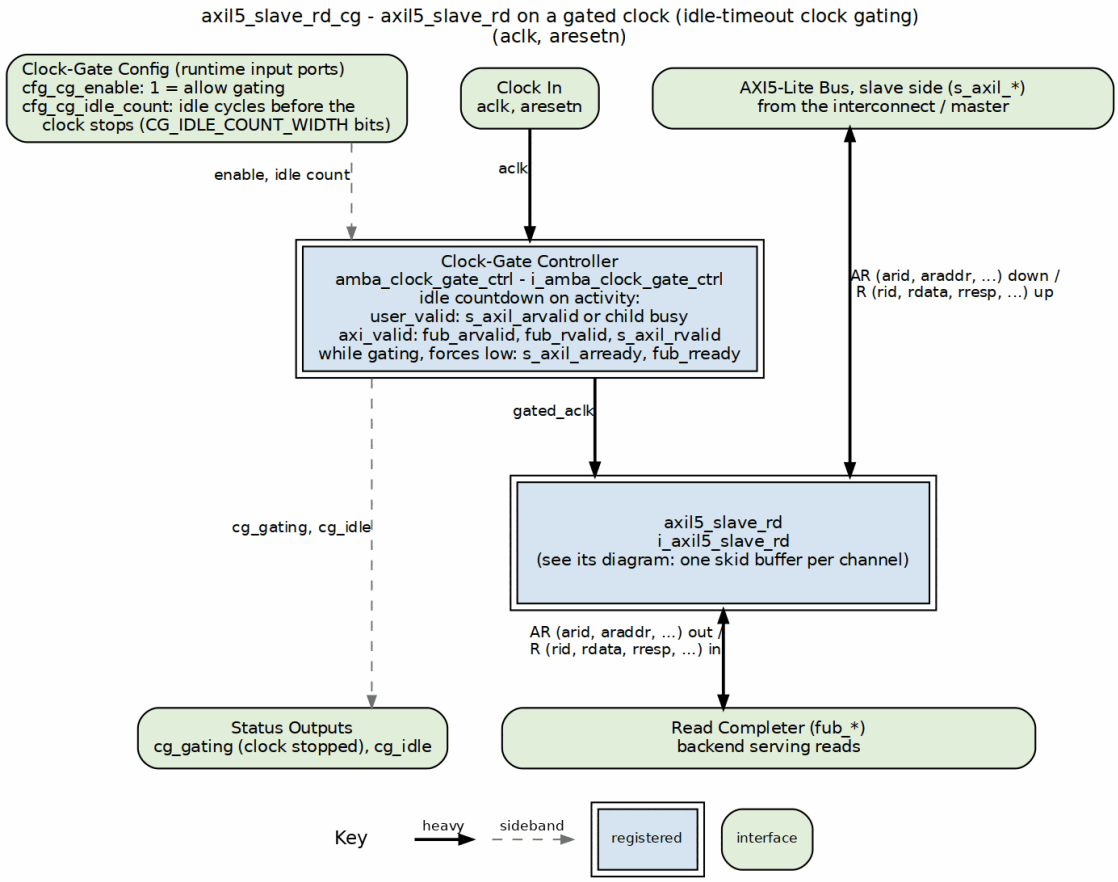
The `s_axil_*` side faces the bus; your backend drives the `fub_*` side. The `cfg_cg_*` inputs run the clock gate.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cfg_cg_enable`	Input	1	
`cfg_cg_idle_count`	Input	`[CG_IDLE_COUNT_WIDTH-1:0]`	
`s_axil_arid`	Input	`[IW-1:0]`	
`s_axil_araddr`	Input	`[AW-1:0]`	
`s_axil_arprot`	Input	`[2:0]`	
`s_axil_aruser`	Input	`[UW-1:0]`	
`s_axil_artrace`	Input	1	
`s_axil_arvalid`	Input	1	
`s_axil_arready`	Output	1	
`fub_arid`	Output	`[IW-1:0]`	
`fub_araddr`	Output	`[AW-1:0]`	
`fub_arprot`	Output	`[2:0]`	
`fub_aruser`	Output	`[UW-1:0]`	
`fub_artrace`	Output	1	
`fub_arvalid`	Output	1	
`fub_arready`	Input	1	
`fub_rid`	Input	`[IW-1:0]`	
`fub_rdata`	Input	`[DW-1:0]`	
`fub_rresp`	Input	`[1:0]`	
`fub_ruser`	Input	`[UW-1:0]`	
`fub_rtrace`	Input	1	
`fub_rpoison`	Input	`[PW-1:0]`	
`fub_rvalid`	Input	1	

Port	Direction	Width	Description
fub_rready	Output	1	
s_axil_rid	Output	[IW-1:0]	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

11.4 Functional Description

11.4.1 Block diagram



axil5_slave_rd_cg block diagram

`axil5_slave_rd` is one child box (see its own diagram), clocked by `gated_aclk`. The heavy clock column runs from `aclk` through `amba_clock_gate_ctrl`, which counts down an idle window and stops the clock when nothing is active; its label lists the exact `user_valid` and `axi_valid` activity terms and the readies it forces low while gating, so no transfer is accepted on a stopped clock. `cg_gating` and `cg_idle` leave as status outputs.

11.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32;

Group	Parameter	Signals on this module	Width
	enable)		>= 1 on slave-side modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

11.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

11.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given

leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the “tied LOW” of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

11.5 Timing Characteristics

11.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

11.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

11.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_cg #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4),
    .CG_IDLE_COUNT_WIDTH (4)
) u_axil5_slave_rd_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // configuration
    .cfg_cg_enable       (cfg_cg_enable),
    .cfg_cg_idle_count   (cfg_cg_idle_count),
    // s_axil_ar
    .s_axil_arid         (s_axil_arid),
    // ... `s_axil_araddr`, `s_axil_arprot`, `s_axil_aruser`, +3 more
    // fub_ar
    .fub_arid            (fub_arid),
    // ... `fub_araddr`, `fub_arprot`, `fub_aruser`, +3 more
    // fub_r
    .fub_rid             (fub_rid),
    // ... `fub_rdata`, `fub_rresp`, `fub_ruser`, +4 more
    // s_axil_r
    .s_axil_rid          (s_axil_rid),
    // ... `s_axil_rdata`, `s_axil_rresp`, `s_axil_ruser`, +4 more
    // status
    .cg_gating           (cg_gating),
    .cg_idle             (cg_idle)
);
```

11.7 Design Notes

No ID and all optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are sums of ID_WIDTH and conditional terms over the ENABLE_* parameters. With ID_WIDTH=0 (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts against both the AXI4-Lite RTL and axil5_master_rd configured that way.

Illegal widths do not elaborate. An initial begin : param_check block under `ifndef SYNTHESIS stops the build when AXIL_DATA_WIDTH is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or ID_WIDTH is outside 0..32 (1..32 on the slave-side modules). val/amba/test_axil_param_guard.py checks it.

Do not override a derived parameter. The *Size parameters and the AW/ DW/IW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The _mon/_monlite wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module's busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches amba_clock_gate_ctrl only. With it low the clock free-runs and the module behaves exactly like its base.

11.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

11.9 Testing

`val/amba/test_axil5_slave_rd_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

11.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

11.11 Navigation

AXI5-Lite index | AMBA overview

12 AXIL5 Slave Read Monitor

Module: `axil5_slave_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon`

12.1 Overview

The AXIL5 Slave Read Monitor provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)
- Integrated transaction monitor emitting monbus packets

12.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-

Parameter	Type	Default	Description
			slice filter, inherited from the shared monitor core. Since amba #104 the wrapper feeds the AXI5-Lite ID, so the slice selects on AxID as on the AXI4 wrappers.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE=1. Compared against the AXI5-Lite AxID.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0014	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)

Parameter	Type	Default	Description
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

12.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_axil_* side. The cfg_* and monbus_* ports belong to the monitor.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_arid	Input	[IW-1:0]	
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
s_axil_rid	Output	[IW-1:0]	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
fub_axil_arid	Output	[IW-1:0]	
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	

Port	Direction	Width	Description
fub_axil_rid	Input	[IW-1:0]	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts

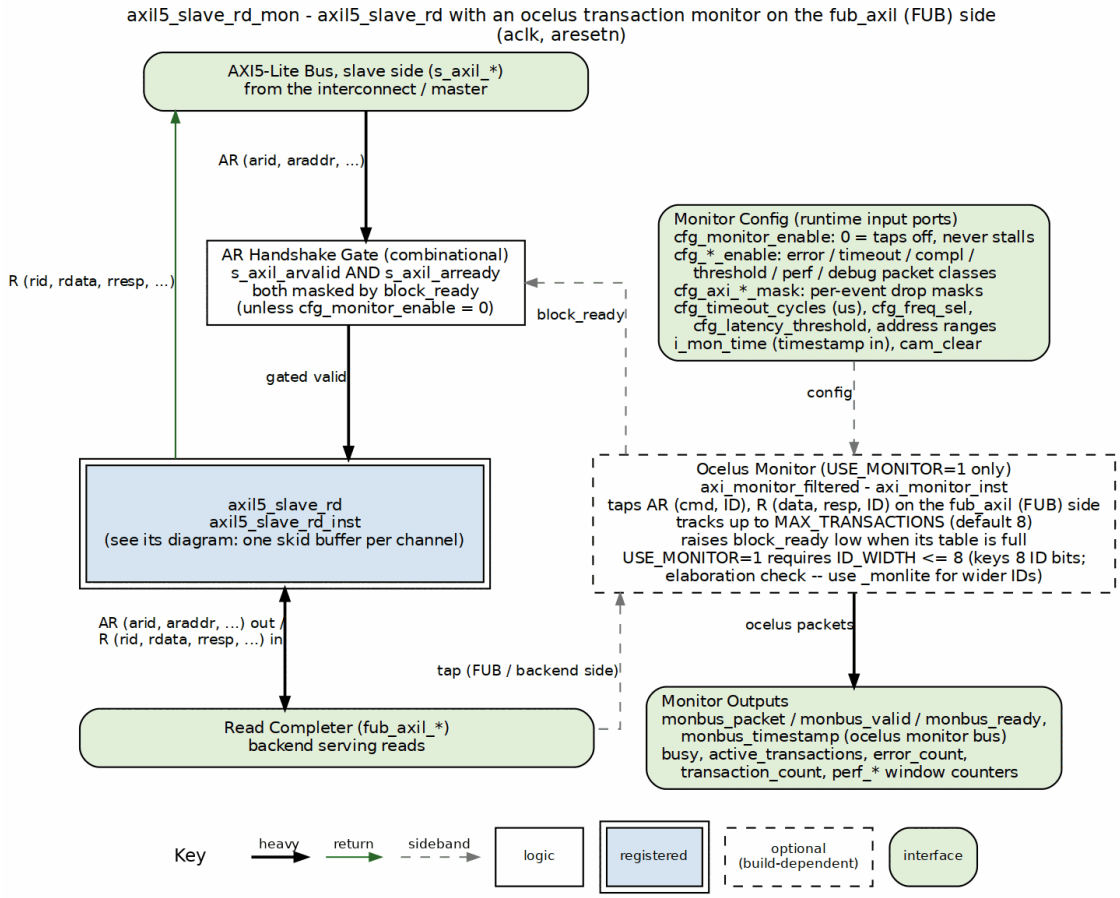
Port	Direction	Width	Description
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid

Port	Direction	Width	Description
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transacti	Output	[7:0]	Number of active

Port	Direction	Width	Description
ons			transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

12.4 Functional Description

12.4.1 Block diagram



axil5_slave_rd_mon block diagram

`axil5_slave_rd` is one child box (see its own diagram). The AR request first passes a handshake gate that masks valid and ready with `block_ready`, so a full monitor table stalls new requests instead of losing them. The optional ocelus monitor (`axi_monitor_filtered`, dashed because `USE_MONITOR=0` removes it) taps the FUB-side (`fub_axil`) backend handshake below the core (every monitor taps the FUB side, for timing) while `block_ready` still gates the `s_axil` handshake, so the watchpoint trails the gate by the skid depth that the reserve margin covers; it carries the transaction IDs and emits monitor-bus packets as the heavy edge. With `USE_MONITOR=1` the build requires `ID_WIDTH <= 8`; the `_monlite` wrapper takes wider IDs.

12.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	ID_WIDTH (no enable)	AxID, BID, RID	ID_WIDTH (0..32; >= 1 on slave-side modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

12.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

12.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the

Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them "not present" on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

12.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

12.6 Timing Characteristics

12.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

12.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

12.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_mon #(
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ACLK_MHZ            (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .UNIT_ID             (8'h02),
    .AGENT_ID            (16'h0014),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_slave_rd_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+31 more
    // s_axil_ar
    .s_axil_arid         (s_axil_arid),
    // ... `s_axil_araddr`, `s_axil_arprot`, `s_axil_aruser`, +3 more
```

```
    // s_axil_r
    .s_axil_rid          (s_axil_rid),
    // ... `s_axil_rdata`, `s_axil_rresp`, `s_axil_ruser`, +4 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

12.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

12.9 Testing

`val/amba/test_axil5_slave_rd_mon.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

12.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

12.11 Navigation

AXI5-Lite index | AMBA overview

13 AXIL5 Slave Read Monitor, Clock-Gated

Module: `axil5_slave_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon_cg`

13.1 Overview

The AXIL5 Slave Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

13.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axil5_slave_rd_monlite_cg`: the same gating logic around `axil5_slave_rd_monlite` instead of `axil5_slave_rd_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	`2`	
`SKID_DEPTH_R`	int	`4`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor in inner mon; outputs tied
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`ADDR_FILTER_ENABLE`	bit	`1'b0`	0 = address filter inert
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions (reduced for

Parameter	Type	Default	Description
			AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0

Parameter	Type	Default	Description
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0014	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

13.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_axil_* side. The cfg_* and monbus_* ports belong to the monitor, and cfg_cg_* runs the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_arid	Input	[IW-1:0]	
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
s_axil_rid	Output	[IW-1:0]	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
fub_axil_arid	Output	[IW-1:0]	
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	

Port	Direction	Width	Description
fub_axil_rid	Input	[IW-1:0]	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts

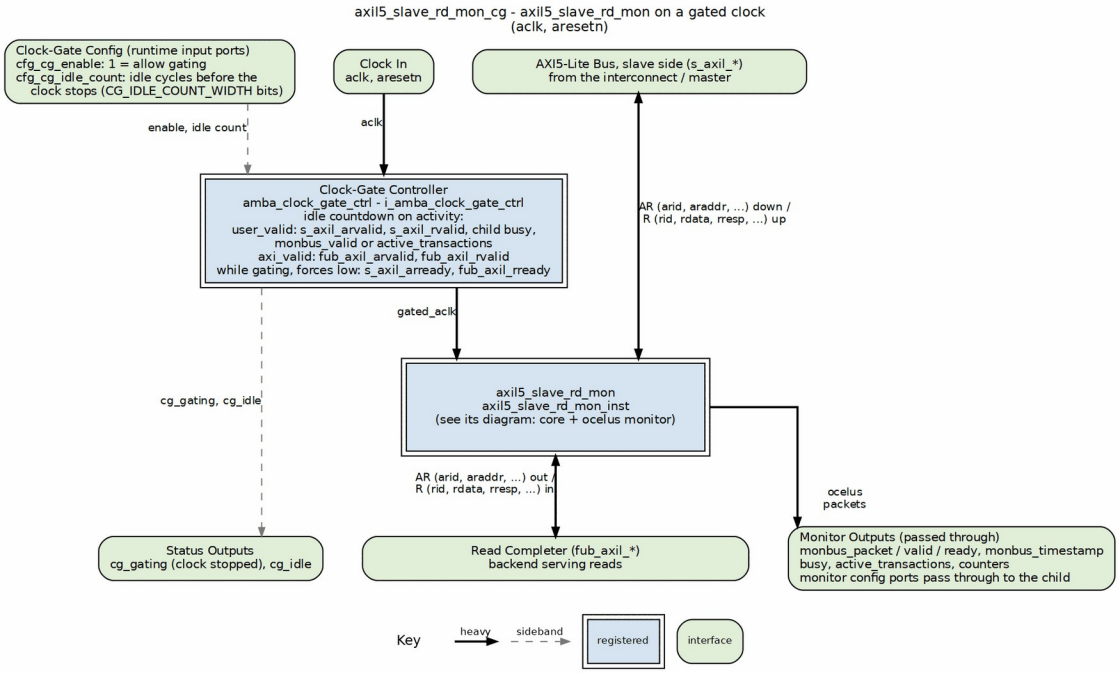
Port	Direction	Width	Description
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	

Port	Direction	Width	Description
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs

Port	Direction	Width	Description
			with `cfg_addr_range_low`.
`busy`	Output	1	
`active_transactions`	Output	`[7:0]`	Number of active transactions
`error_count`	Output	`[15:0]`	Total error count
`transaction_count`	Output	`[31:0]`	Total transaction count
`cg_gating`	Output	1	Gated clock is stopped
`cg_idle`	Output	1	No activity observed
`cfg_conflict_error`	Output	1	Configuration conflict detected
`cfg_start_event_sel`	Input	`[2:0]`	
`cfg_end_event_sel`	Input	`[2:0]`	
`cfg_start_trigger`	Input	1	
`cfg_end_trigger`	Input	1	
`cfg_window_force_close`	Input	1	
`window_active`	Output	1	
`window_cycles`	Output	`[31:0]`	
`perf_prod_cycles`	Output	`[31:0]`	
`perf_bp_cycles`	Output	`[31:0]`	
`perf_starv_cycles`	Output	`[31:0]`	
`perf_idle_cycles`	Output	`[31:0]`	
`perf_beat_count`	Output	`[31:0]`	
`perf_byte_count`	Output	`[63:0]`	
`perf_burst_count`	Output	`[31:0]`	

13.4 Functional Description

13.4.1 Block diagram



axil5_slave_rd_mon_cg block diagram

`axil5_slave_rd_mon` (core plus ocelus monitor) is one child box (see its own diagram), clocked by `gated_aclk` from `amba_clock_gate_ctrl`. The controller's label lists the activity terms, which add `monbus_valid` and `active_transactions` so the clock keeps running while the monitor still has packets or open transactions. Monitor-bus packets and the monitor configuration pass straight through to the child.

13.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`

Group	Parameter	Signals on this module	Width
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

13.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

13.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

13.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

13.6 Timing Characteristics

13.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

13.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

13.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_mon_cg #(
    .SKID_DEPTH_AR       (2),
    .SKID_DEPTH_R        (4),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ACLK_MHZ            (100),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .UNIT_ID             (8'h02),
    .AGENT_ID            (16'h0014),
    .MAX_TRANSACTIONS    (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .CG_IDLE_COUNT_WIDTH (4),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_slave_rd_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+32 more
    // s_axil_ar
    .s_axil_arid         (s_axil_arid),
    // ... `s_axil_araddr`, `s_axil_arprot`, `s_axil_aruser`, +3 more
    // s_axil_r
    .s_axil_rid          (s_axil_rid),
```

```
    // ... `s_axil_rdata`, `s_axil_rresp`, `s_axil_ruser`, +4 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

13.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

13.9 Testing

`val/amba/test_axil5_slave_rd_mon_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

13.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AR and R channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- READY held LOW while gated is legal: a receiver may keep READY LOW until it chooses to accept (IHI 0022H §A3.2.1; IHI 0022L §A2.3.1). No paper (standard technique).
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

13.11 Navigation

AXI5-Lite index | AMBA overview

14 AXIL5 Slave Write

Module: `axil5_slave_wr.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr`

14.1 Overview

The AXIL5 Slave Write provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)

14.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ?	

Parameter	Type	Default	Description
		`(DW / 64) : 1`	
`AWSize`	int	`AW + 3 +`	
`WSize`	int	`DW + SW +`	
`BSize`	int	`2 +`	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

14.3 Ports

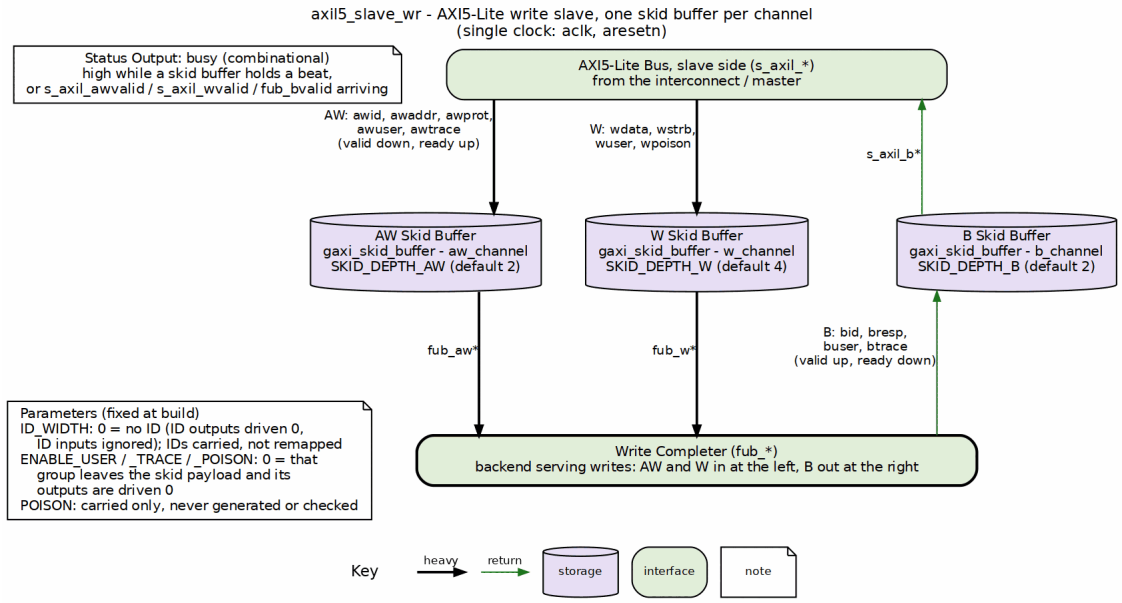
The `s_axil_*` side faces the bus; your backend drives the `fub_*` side.

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`s_axil_awid`	Input	`[IW-1:0]`	
`s_axil_awaddr`	Input	`[AW-1:0]`	
`s_axil_awprot`	Input	`[2:0]`	
`s_axil_awuser`	Input	`[UW-1:0]`	
`s_axil_awtrace`	Input	1	
`s_axil_awvalid`	Input	1	
`s_axil_awready`	Output	1	
`fub_awid`	Output	`[IW-1:0]`	
`fub_awaddr`	Output	`[AW-1:0]`	
`fub_awprot`	Output	`[2:0]`	
`fub_awuser`	Output	`[UW-1:0]`	
`fub_awtrace`	Output	1	
`fub_awvalid`	Output	1	
`fub_awready`	Input	1	
`s_axil_wdata`	Input	`[DW-1:0]`	
`s_axil_wstrb`	Input	`[SW-1:0]`	
`s_axil_wuser`	Input	`[UW-1:0]`	
`s_axil_wpoison`	Input	`[PW-1:0]`	
`s_axil_wvalid`	Input	1	
`s_axil_wready`	Output	1	
`fub_wdata`	Output	`[DW-1:0]`	
`fub_wstrb`	Output	`[SW-1:0]`	
`fub_wuser`	Output	`[UW-1:0]`	

Port	Direction	Width	Description
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bid	Input	[IW-1:0]	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bid	Output	[IW-1:0]	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
busy	Output	1	

14.4 Functional Description

14.4.1 Block diagram



axil5_slave_wr block diagram

The AXI5-Lite bus (`s_axil_*`) sits at the top and the completer (`fub_*`) is the bar at the bottom. AW and W run down through their own `gaxi_skid_buffers`, and the B response climbs back up its own lane on the right through a third, so every channel crosses exactly one buffer. This is the axil4 structure; AXI5-Lite only adds carried fields – the ID on every channel and the optional USER, TRACE and POISON groups, each dropped from the skid payload (outputs driven 0) when its `ENABLE_*` parameter is 0, as the Parameters legend states. `busy` is high while any skid buffer holds a beat or a valid is arriving (s_axil_awvalid, s_axil_wvalid, fub_bvalid), the same definition as every other AMBA base module.

14.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)
USER	`ENABLE_USER`	`AxUSER`, `WUSER`,	`USER_WIDTH`

Group	Parameter	Signals on this module	Width
		BUSER, RUSER	
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

14.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

14.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

14.5 Timing Characteristics

14.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

14.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

14.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (4),
    .SKID_DEPTH_B        (2)
) u_axil5_slave_wr (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // s_axil_aw
    .s_axil_awid         (s_axil_awid),
    // ... `s_axil_awaddr`, `s_axil_awprot`, `s_axil_awuser`, +3 more
    // fub_aw
    .fub_awid            (fub_awid),
    // ... `fub_awaddr`, `fub_awprot`, `fub_awuser`, +3 more
    // s_axil_w
    .s_axil_wdata        (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // fub_w
    .fub_wdata           (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // fub_b
    .fub_bid             (fub_bid),
    // ... `fub_bresp`, `fub_buser`, `fub_btrace`, +2 more
    // s_axil_b
    .s_axil_bid          (s_axil_bid),
    // ... `s_axil_bresp`, `s_axil_buser`, `s_axil_btrace`, +2 more
    // status
    .busy                (busy)
);
```

14.7 Design Notes

No ID and all optional groups off is AXI4-Lite. `ARSize`/`AWSize`/`WSize`/`RSize`/ `BSize` are sums of `ID_WIDTH` and conditional terms over the `ENABLE_*` parameters. With `ID_WIDTH=0` (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what `val/amba/test_axil5_axil4_equivalence.py` asserts against both the AXI4-Lite RTL and `axil5_master_rd` configured that way.

Illegal widths do not elaborate. An `initial begin : param_check` block under `` `ifndef SYNTHESIS `` stops the build when `AXIL_DATA_WIDTH` is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or `ID_WIDTH` is outside 0..32 (1..32 on the slave-side modules). `val/amba/test_axil_param_guard.py` checks it.

Do not override a derived parameter. The `*Size` parameters and the `AW`/ `DW`/`IW` aliases are declared as `parameter` so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The `_mon`/`_monlite` wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

14.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr` – the AXI4-Lite counterpart
- AXI4-Lite modules

14.9 Testing

`val/amba/test_axil5_slave_wr.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

14.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: storage on a back-pressured channel adds latency without changing function.
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Bruno, F., Eschemann, G. *The FPGA Programming Handbook*, 2nd ed., Packt, 2024, ch. 8 “Introduction to AXI”, §8.3 “AXI4 interfaces (full and AXI-Lite)” – textbook treatment of the AXI-Lite channel set (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

14.11 Navigation

AXI5-Lite index | AMBA overview

15 AXIL5 Slave Write, Clock-Gated

Module: `axil5_slave_wr_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_cg`

15.1 Overview

The AXIL5 Slave Write, Clock-Gated provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_cg` with the ID and those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)
- One `amba_clock_gate_ctrl` gating the whole inner module

15.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0

Parameter	Type	Default	Description
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

15.3 Ports

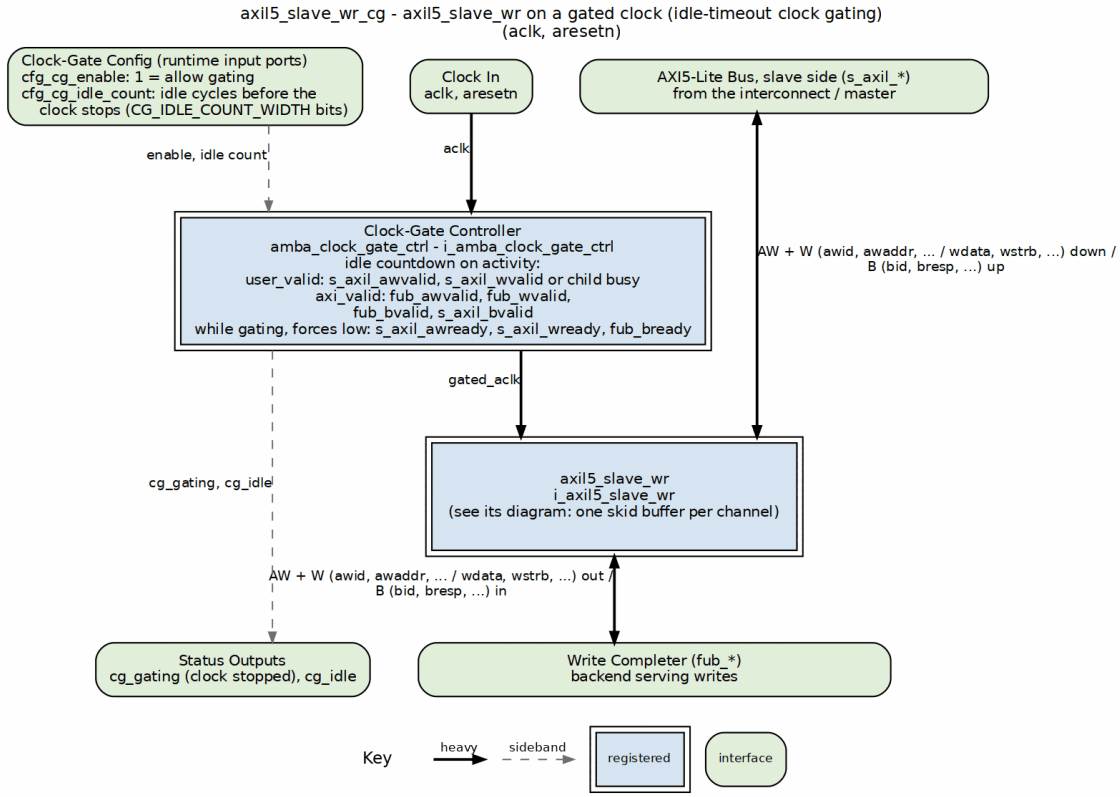
The s_axil_* side faces the bus; your backend drives the fub_* side. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axil_awid	Input	[IW-1:0]	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
fub_awid	Output	[IW-1:0]	
fub_awaddr	Output	[AW-1:0]	
fub_awprot	Output	[2:0]	
fub_awuser	Output	[UW-1:0]	
fub_awtrace	Output	1	
fub_awvalid	Output	1	
fub_awready	Input	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	

Port	Direction	Width	Description
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bid	Input	[IW-1:0]	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bid	Output	[IW-1:0]	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

15.4 Functional Description

15.4.1 Block diagram



axil5_slave_wr_cg block diagram

`axil5_slave_wr` is one child box (see its own diagram), clocked by `gated_aclk`. The heavy clock column runs from `aclk` through `amba_clock_gate_ctrl`, which counts down an idle window and stops the clock when nothing is active; its label lists the exact `user_valid` and `axi_valid` activity terms and the readies it forces low while gating, so no transfer is accepted on a stopped clock. `cg_gating` and `cg_idle` leave as status outputs.

15.4.2 Optional signal groups

The ID plus three optional groups – the ones IHI 0022L Table B2.2 allows on AXI5-Lite. The ID has no enable (it is sized by `ID_WIDTH`); each group is gated by its own `ENABLE_*` parameter and contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

15.4.3 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

15.4.4 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

15.5 Timing Characteristics

15.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

15.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

15.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_cg #(
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1),
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (4),
    .SKID_DEPTH_B        (2),
    .CG_IDLE_COUNT_WIDTH (4)
) u_axil5_slave_wr_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // configuration
    .cfg_cg_enable       (cfg_cg_enable),
    .cfg_cg_idle_count   (cfg_cg_idle_count),
    // s_axil_aw
    .s_axil_awid         (s_axil_awid),
    // ... `s_axil_awaddr`, `s_axil_awprot`, `s_axil_awuser`, +3 more
    // fub_aw
    .fub_awid            (fub_awid),
    // ... `fub_awaddr`, `fub_awprot`, `fub_awuser`, +3 more
    // s_axil_w
    .s_axil_wdata        (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // fub_w
    .fub_wdata           (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // fub_b
    .fub_bid             (fub_bid),
    // ... `fub_bresp`, `fub_buser`, `fub_btrace`, +2 more
    // s_axil_b
    .s_axil_bid          (s_axil_bid),
    // ... `s_axil_bresp`, `s_axil_buser`, `s_axil_btrace`, +2 more
    // status
    .cg_gating           (cg_gating),
    .cg_idle             (cg_idle)
);
```

15.7 Design Notes

No ID and all optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are sums of ID_WIDTH and conditional terms over the ENABLE_* parameters. With ID_WIDTH=0 (Manager side only) and every group disabled they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts against both the AXI4-Lite RTL and axil5_master_rd configured that way.

Illegal widths do not elaborate. An initial begin : param_check block under `ifndef SYNTHESIS stops the build when AXIL_DATA_WIDTH is not an AXI data width (8..1024, power of two; IHI 0022L Table A3.19 – Table B2.4 adds no AXI5-Lite restriction) or ID_WIDTH is outside 0..32 (1..32 on the slave-side modules). val/amba/test_axil_param_guard.py checks it.

Do not override a derived parameter. The *Size parameters and the AW/ DW/IW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads "Extracting 4 bits from only 2 bit number", and it is how this family's test port broke.

The ID is AXI5-Lite's, not the monitor's. AXI5-Lite carries AWID/BID and ARID/RID (IHI 0022L Table B2.2, code YS: optional on a Manager, mandatory on a Subordinate) and this family carries them through. The _mon/_monlite wrappers feed it to the shared monitor core (amba #104), so each response is paired with the oldest outstanding command of its own ID and a Subordinate may reorder across IDs (B2.1.5); W carries no ID and is attributed in AW order (A5.5).

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module's busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches amba_clock_gate_ctrl only. With it low the clock free-runs and the module behaves exactly like its base.

15.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

15.9 Testing

`val/amba/test_axil5_slave_wr_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

15.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

15.11 Navigation

AXI5-Lite index | AMBA overview

16 AXIL5 Slave Write Monitor

Module: `axil5_slave_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon`

16.1 Overview

The AXIL5 Slave Write Monitor gives a slave device a buffered AXI5-Lite write interface — and keeps a transaction monitor on the path while it's at it.

If you take one thing from this page, take this: AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon` with the ID and those groups threaded through the packed SKID payload. Know the AXI4-Lite part and you know this one.

16.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)
- Integrated transaction monitor emitting monbus packets

16.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor, tie outputs
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions (reduced for AXIL)
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	

Parameter	Type	Default	Description
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Since amba #104 the wrapper feeds the AXI5-Lite ID, so the slice selects on AxID as on the AXI4 wrappers.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE=1. Compared against the AXI5-Lite AxID.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0015	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-

Parameter	Type	Default	Description
			bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Force a *Size to a value the RTL didn't derive and every optional-group field turns into a part-select past the end of the vector — which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

16.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awid	Input	[IW-1:0]	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bid	Output	[IW-1:0]	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awid	Output	[IW-1:0]	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	

Port	Direction	Width	Description
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bid	Input	[IW-1:0]	
fub_axil_bresp	Input	[1:0]	
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in

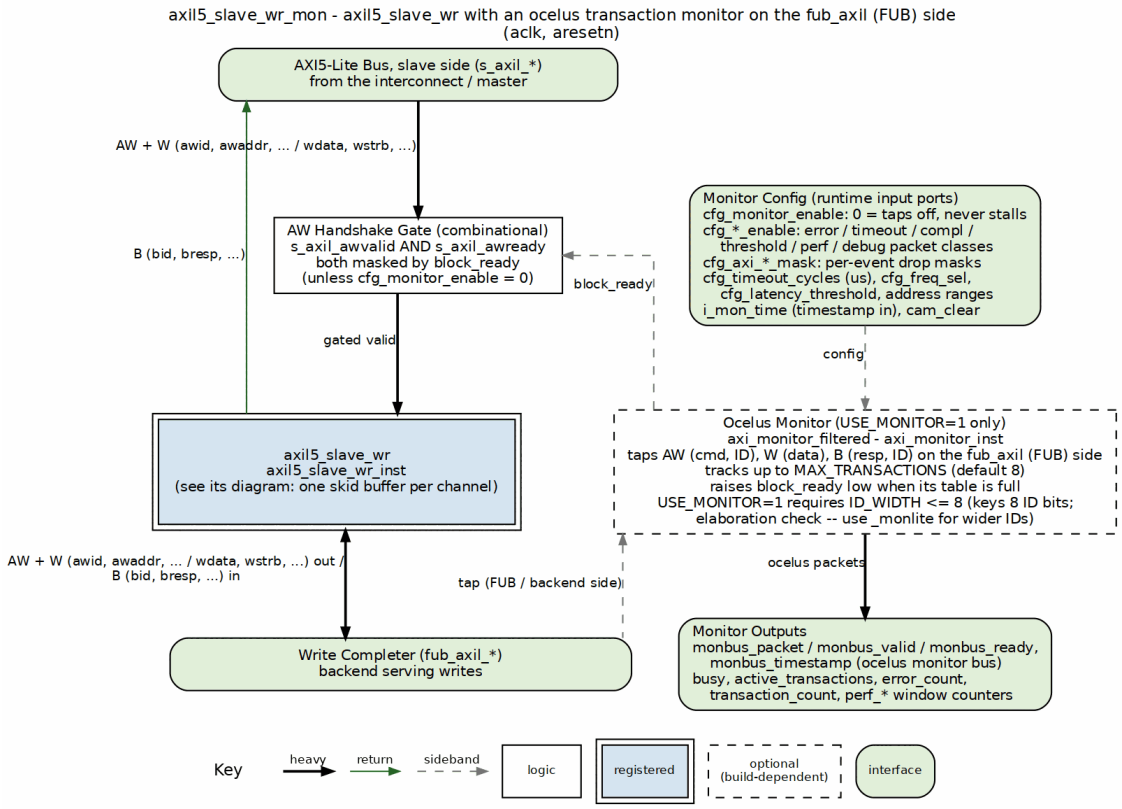
Port	Direction	Width	Description
			MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES :	

Port	Direction	Width	Description
		1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x	High bound of

Port	Direction	Width	Description
		N_ADDR_RANGES	each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

16.4 Functional Description

16.4.1 Block diagram



axil5_slave_wr_mon block diagram

`axil5_slave_wr` is one child box (see its own diagram). The AW request first passes a handshake gate that masks valid and ready with `block_ready`, so a full monitor table stalls new requests instead of losing them. The optional ocelus monitor (`axi_monitor_filtered`, dashed because `USE_MONITOR=0` removes it) taps the FUB-side (`fub_axil`) backend handshake below the core (every monitor taps the FUB side, for timing) while `block_ready` still gates the `s_axil` handshake, so the watchpoint trails the gate by the skid depth that the reserve margin covers; it carries the transaction IDs and emits monitor-bus packets as the heavy edge. With `USE_MONITOR=1` the build requires `ID_WIDTH <= 8`; the `_monlite` wrapper takes wider IDs.

AXI5-Lite adds the transaction ID (`ID_WIDTH`, no enable; mandatory on this Subordinate-side module) and three optional signal groups – USER, TRACE and POISON, the ones IHI 0022L Table B2.2 allows – each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when it's enabled — turn it off and those bits simply aren't in the vector.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32;

Group	Parameter	Signals on this module	Width
	enable)		>= 1 on slave-side modules)
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

16.4.2 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

16.4.3 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given

leaves on the other side. With `ID_WIDTH = 0` (Manager-side modules only) the ID outputs are driven 0 – the “tied LOW” of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

16.5 Design Notes

A handful of traps specific to the monitored variants. Each one is written down because someone hit it.

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong — silently. That's the worst kind of wrong.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination. At least that one fails loud.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

16.6 Timing Characteristics

16.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

16.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

16.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_mon #(
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (2),
    .SKID_DEPTH_B        (2),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .ACLK_MHZ            (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .UNIT_ID             (8'h02),
    .AGENT_ID            (16'h0015),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_slave_wr_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+35 more
    // s_axil_aw
    .s_axil_awid         (s_axil_awid),
```

```
    // ... `s_axil_awaddr`, `s_axil_awprot`, `s_axil_awuser`, +3 more
    // s_axil_w
    .s_axil_wdata        (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // s_axil_b
    .s_axil_bid          (s_axil_bid),
    // ... `s_axil_bresp`, `s_axil_buser`, `s_axil_btrace`, +2 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

16.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

16.9 Testing

`val/amba/test_axil5_slave_wr_mon.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** — `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake — and the version you'd rather get.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically. That's exactly how you want a variant family to work.

16.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conf. (DAC ’06)*, pp. 657-662, 2006. DOI: 10.1145/1146909.1147077 – the elastic buffer; its Property 3.1 (capacity C >= 2 for full throughput) is the depth-2 floor of each channel’s `gaxi_skid_buffer`.
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Chipressian. *Crack the Hardware Interview – from RTL Designers’ Perspective: Architecture and Micro-architecture Design*, ch. 2 “Handshake Protocols”, Q8 valid-ready protocol and Q9 valid-ready slices – textbook treatment of register slices (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `ElasticBuffer__Cortadella2006.pdf`)

16.11 Navigation

AXI5-Lite index | AMBA overview

17 AXIL5 Slave Write Monitor, Clock-Gated

Module: `axil5_slave_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon_cg`

17.1 Overview

The AXIL5 Slave Write Monitor, Clock-Gated gives a slave device a buffered AXI5-Lite write interface — the same monitor as the plain variant, with clock gating stacked on top and the ready signals held low while gated.

If you take one thing from this page, take this: AXI5-Lite is AXI4-Lite plus the transaction ID and a few optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon_cg` with the ID and those groups threaded through the packed SKID payload. Know the AXI4-Lite part and you know this one.

17.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Elaboration fails on an illegal `AXIL_DATA_WIDTH` or `ID_WIDTH`
- Transaction ID (`AWID`/`BID`, `ARID`/`RID`; `ID_WIDTH`) plus three optional signal groups (USER, TRACE, POISON), each independently enabled
- With all groups disabled, the AXI4-Lite module of the same name plus the ID (mandatory on a Subordinate)
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

17.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axil5_slave_wr_monlite_cg`: the same gating logic around `axil5_slave_wr_monlite` instead of `axil5_slave_wr_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	Any AXI data width, 8..1024 and a power of two (IHI 0022L Table A3.19; Table B2.4 adds no AXI5-Lite limit). Checked at elaboration.
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor in inner mon; outputs tied
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`ADDR_FILTER_ENABLE`	bit	`1'b0`	0 = address filter inert
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_WDATA_ORDER_Q`	bit	`1'b0`	
`NUM_BANKS`	int	`1`	
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions

Parameter	Type	Default	Description
			(reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
ID_WIDTH	int	4	AWID/BID and ARID/RID width, 1..32 (Subordinate side: the ID is mandatory, Table B2.2 code YS). IHI 0022L Table A5.2.
USER_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_POISON	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
IW	int	(ID_WIDTH > 0) ? ID_WIDTH : 1	ID port width; a 1-bit placeholder

Parameter	Type	Default	Description
			when ID_WIDTH = 0
UW	int	USER_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0015	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Force a *Size to a value the RTL didn't derive and every optional-group field turns into a part-select past the end of the vector — which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

17.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awid	Input	[IW-1:0]	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bid	Output	[IW-1:0]	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awid	Output	[IW-1:0]	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	

Port	Direction	Width	Description
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bid	Input	[IW-1:0]	
fub_axil_bresp	Input	[1:0]	
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in

Port	Direction	Width	Description
			MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating

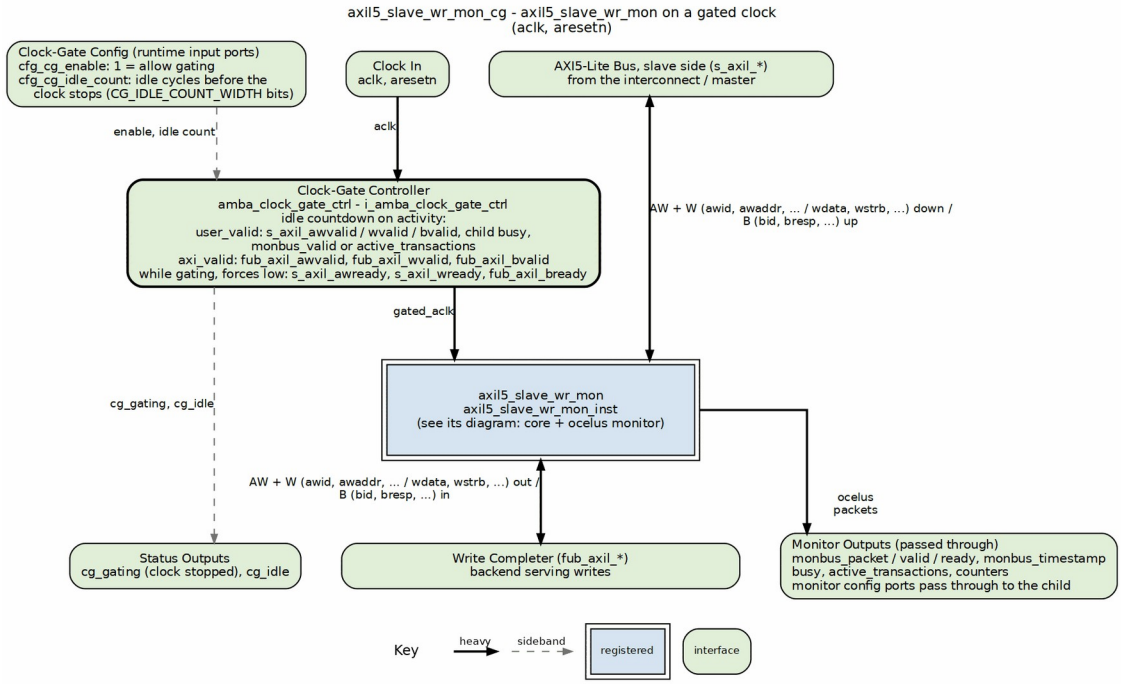
Port	Direction	Width	Description
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only

Port	Direction	Width	Description
			present when `N_ADDR_RANGES > 0`.
`cfg_addr_range_high`	Input	`[AW-1:0]` x `N_ADDR_RANGES`	High bound of each address-range comparator; pairs with `cfg_addr_range_low`.
`busy`	Output	1	
`active_transactions`	Output	`[7:0]`	Number of active transactions
`error_count`	Output	`[15:0]`	Total error count
`transaction_count`	Output	`[31:0]`	Total transaction count
`cg_gating`	Output	1	Gated clock is stopped
`cg_idle`	Output	1	No activity observed
`cfg_conflict_error`	Output	1	Configuration conflict detected
`cfg_start_event_sel`	Input	`[2:0]`	
`cfg_end_event_sel`	Input	`[2:0]`	
`cfg_start_trigger`	Input	1	
`cfg_end_trigger`	Input	1	
`cfg_window_force_close`	Input	1	
`window_active`	Output	1	
`window_cycles`	Output	`[31:0]`	
`perf_prod_cycles`	Output	`[31:0]`	
`perf_bp_cycles`	Output	`[31:0]`	

Port	Direction	Width	Description
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

17.4 Functional Description

17.4.1 Block diagram



axil5_slave_wr_mon_cg block diagram

`axil5_slave_wr_mon` (core plus ocelus monitor) is one child box (see its own diagram), clocked by `gated_aclk` from `amba_clock_gate_ctrl`. The controller's label lists the activity terms, which add `monbus_valid` and `active_transactions` so the clock keeps running while the monitor still has packets or open transactions. Monitor-bus packets and the monitor configuration pass straight through to the child.

AXI5-Lite adds the transaction ID (`ID_WIDTH`, no enable; mandatory on this Subordinate-side module) and three optional signal groups – USER, TRACE and POISON, the ones IHI 0022L Table B2.2 allows – each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when it's enabled — turn it off and those bits simply aren't in the vector.

Group	Parameter	Signals on this module	Width
ID	`ID_WIDTH` (no enable)	`AxID`, `BID`, `RID`	`ID_WIDTH` (0..32; >= 1 on slave-side modules)
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`
TRACE	`ENABLE_TRACE`	`AxTRACE`, `BTRACE`,	1

Group	Parameter	Signals on this module	Width
POISON	ENABLE_POISON	RTRACE WPOISON, RPOISON	one bit per 64 data bits

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

17.4.2 With no ID and every group disabled, this IS the AXI4-Lite module

The packed payload width of a no-ID, fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, ID_WIDTH=0, groups off	AXI5-Lite defaults (ID_WIDTH=4, groups on)
ARSize	35	35	44
RSize	34	34	44
AWSize	35	35	44
WSize	36	36	41
BSize	2	2	11

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default widths. ID_WIDTH = 0 is legal only on the Manager-side axil5_master_* modules; a Subordinate must carry the ID, so the axil5_slave_* modules require ID_WIDTH >= 1 and their minimum payload is the AXI4-Lite one plus the ID on AR/R or AW/B.)

That equivalence is what val/amba/test_axil5_axil4_equivalence.py pins: it runs the same AXI5-Lite BFMs, built with no ID and no groups, against both axil4_master_rd and axil5_master_rd elaborated with ID_WIDTH=0 and every ENABLE_* = 0.

17.4.3 It transports; it does not interpret

The ID is carried in both directions like every other field: AxID crosses with the request, and BID/RID cross back with the response unchanged. Reflecting AxID onto BID/RID is the Subordinate's job (IHI 0022L A5.1.1); this module only guarantees that whatever ID it is given leaves on the other side. With ID_WIDTH = 0 (Manager-side modules only) the ID outputs are driven 0 – the "tied LOW" of A5.1.1 – and the ID inputs are ignored.

TRACE and USER are carried end to end unmodified. POISON is carried – never generated, never checked. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP are not carried. Table B2.2 of IHI 0022L marks them “not present” on AXI5-Lite and B2.1.5 states exclusive accesses are not supported; earlier versions of this module carried them behind `ENABLE_*` parameters (removed by issue #102).

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

17.5 Design Notes

A handful of traps specific to the monitored variants. Each one is written down because someone hit it.

The monitor observes the ID, not the optional groups. Since amba #104 the wrapper feeds AxID and BID/RID to `axi_monitor_filtered`, so a Subordinate that reorders responses across IDs (permitted on AXI5-Lite, IHI 0022L B2.1.5) has each response paired with its own transaction, and a completion packet carries the ID as its channel. W carries no ID and is attributed in AW order (A5.5), including W that arrives ahead of its AW (A2.6). The full monitor keys on at most 8 ID bits, so `ID_WIDTH` > 8 is refused at elaboration here (the `_monlite` wrappers take any width). There are no ports for TRACE, USER or POISON, and POISON is not validated.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong — silently. That's the worst kind of wrong.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination. At least that one fails loud.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

17.6 Timing Characteristics

17.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

17.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

17.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_mon_cg #(
    .SKID_DEPTH_AW       (2),
    .SKID_DEPTH_W        (2),
    .SKID_DEPTH_B        (2),
    .AXIL_ADDR_WIDTH     (32),
    .AXIL_DATA_WIDTH     (32),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES       (0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ACLK_MHZ            (100),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .UNIT_ID             (8'h02),
    .AGENT_ID            (16'h0015),
    .MAX_TRANSACTIONS    (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .CG_IDLE_COUNT_WIDTH (4),
    .ID_WIDTH            (4),
    .USER_WIDTH          (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_POISON       (1)
) u_axil5_slave_wr_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+36 more
    // s_axil_aw
    .s_axil_awid         (s_axil_awid),
    // ... `s_axil_awaddr`, `s_axil_awprot`, `s_axil_awuser`, +3 more
    // s_axil_w
```

```
    .s_axil_wdata        (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // s_axil_b
    .s_axil_bid          (s_axil_bid),
    // ... `s_axil_bresp`, `s_axil_buser`, `s_axil_btrace`, +2 more
    // configuration
    .cfg_monitor_enable  (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
    // monitor bus
    .monbus_valid        (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

17.8 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

17.9 Testing

`val/amba/test_axil5_slave_wr_mon_cg.py` drives this module with the AXI5-Lite BFMs and **the ID and every optional group enabled** — `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake — and the version you'd rather get.

Every transaction the Manager-side BFM issues carries a rotating non-zero ID and must come back with the same BID/RID (`bin/TBClasses/axil5/axil5_id_check.py`); the inherited AXI4-Lite phases alone would only ever send ID 0, which a module that dropped the ID would pass.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically. That's exactly how you want a variant family to work.

17.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/l – §B2.1.5 “AXI5-Lite” and Table B2.2 signal matrix; carried groups per §A12.5 User defined signaling, §A12.3 Trace signals, §A5.1.1 transaction ID signals, §A16.1 Poison; handshake on the AW, W and B channels per §A2.3.1/§A2.3.2; register stages per §A2.5. Table B2.2 marks AxLOCK, AxMPAM, AxMECID, AxNSAID and xLOOP not present on AXI5-Lite and AxID/BID/RID mandatory on a Subordinate (YS): this module carries the ID (`ID_WIDTH`) and none of the not-present signals (issue #102). Data width per Table A3.19 (any AXI DATA_WIDTH; Table B2.4 adds no AXI5-Lite constraint).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, 2020. developer.arm.com/documentation/ihi0022/h – Part C ch. C2 “AMBA AXI5-Lite”, §C2.2 comparison with AXI4-Lite and §C2.5/§C2.6 upgrading an AXI4-Lite master/slave: the `ID_WIDTH=0`, every-group-disabled Manager-side build is the AXI4-Lite-equivalent port.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on any VALID.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based glitch-free clock gate under `clock_gate_ctrl`.
- READY held LOW while gated is legal: a receiver may keep READY LOW until it chooses to accept (IHI 0022H §A3.2.1; IHI 0022L §A2.3.1). No paper (standard technique).
- The monitor itself (`axi_monitor_filtered`) is referenced in the monitor pages; on this port it sees single-beat transactions carrying the AXI5-Lite ID (IHI 0022L B2.1.5; W in AW order, A5.5).
- Roy, S. *Advanced Digital System Design*, Springer, 2024, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of disabling the clock to idle units (secondary source).
- Indexed in `rtl/amba/references/README.md`; the clock-gating papers are archived in `rtl/common/references/` (local PDF: `DynamicPowerManagementTimeout__Benini2000.pdf`, `LatchBasedClockGating__Benini1996.pdf`)

17.11 Navigation

AXI5-Lite index | AMBA overview